COMP1860 Building our Digital World: Computer Systems & Architecture

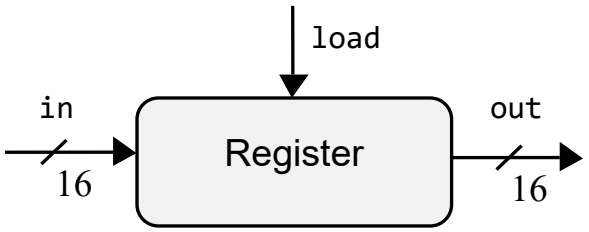
S1.W7.L2: Sequential Logic & Memory

Leandro Soares Indrusiak

Objectives

1. Understand the use of clocks(时钟) in computer systems
2. Explain the difference between Sequential & Combinational logic
3. Understand how Registers work
4. Implement the circuits (CHIPS) required to implement a register and a Program Counter

Memory



Basic abstractions:

- "Loading" a value
- "Storing" a value

1 2 3 4 5 6 7 ... 205 206 207 208 209 210 211 ... time

(cycles)

x = 17 , 17, 17, 17, 17, 17, 17, ... ,

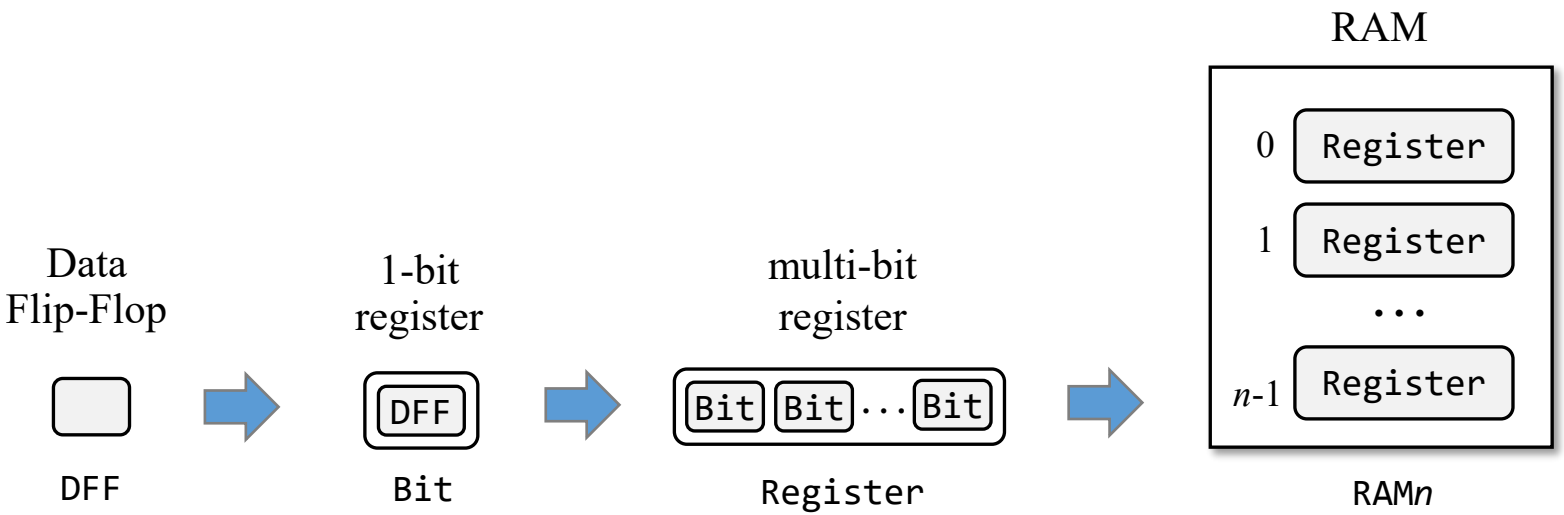
loading storing

x = 21 , 21, 21, 21, 21, 21, 21, ...

loading storing

The challenge: Building chips that realize this functionality.

Memory



The challenge: Building chips that realize this functionality.

Representing time(代表时间)

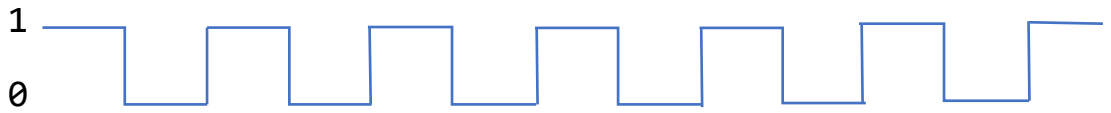
physical time:

Arrow of time:

Continuous（连续的）

Representing time

physical time:

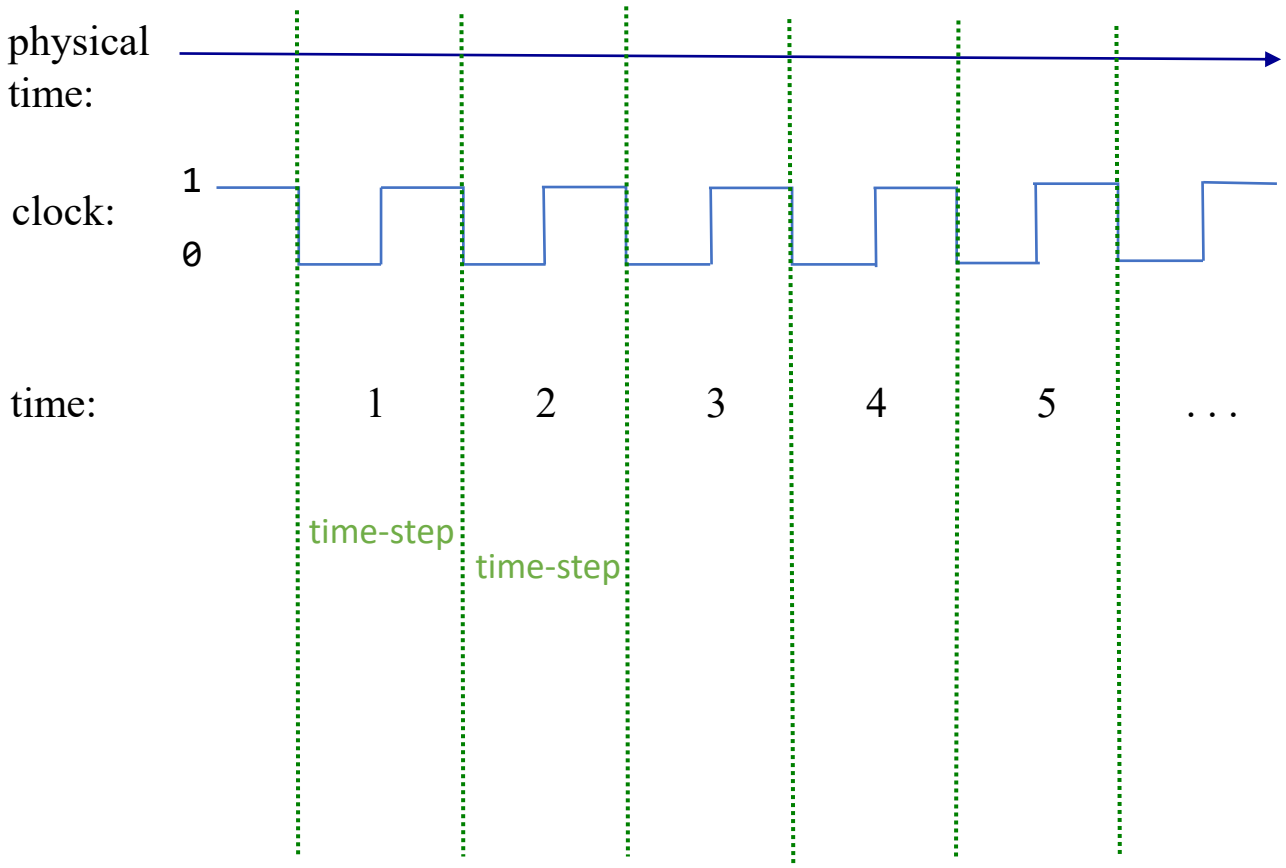


clock:
时钟

Arrow of time:

Continuous

Representing time



Arrow of time:

Continuous

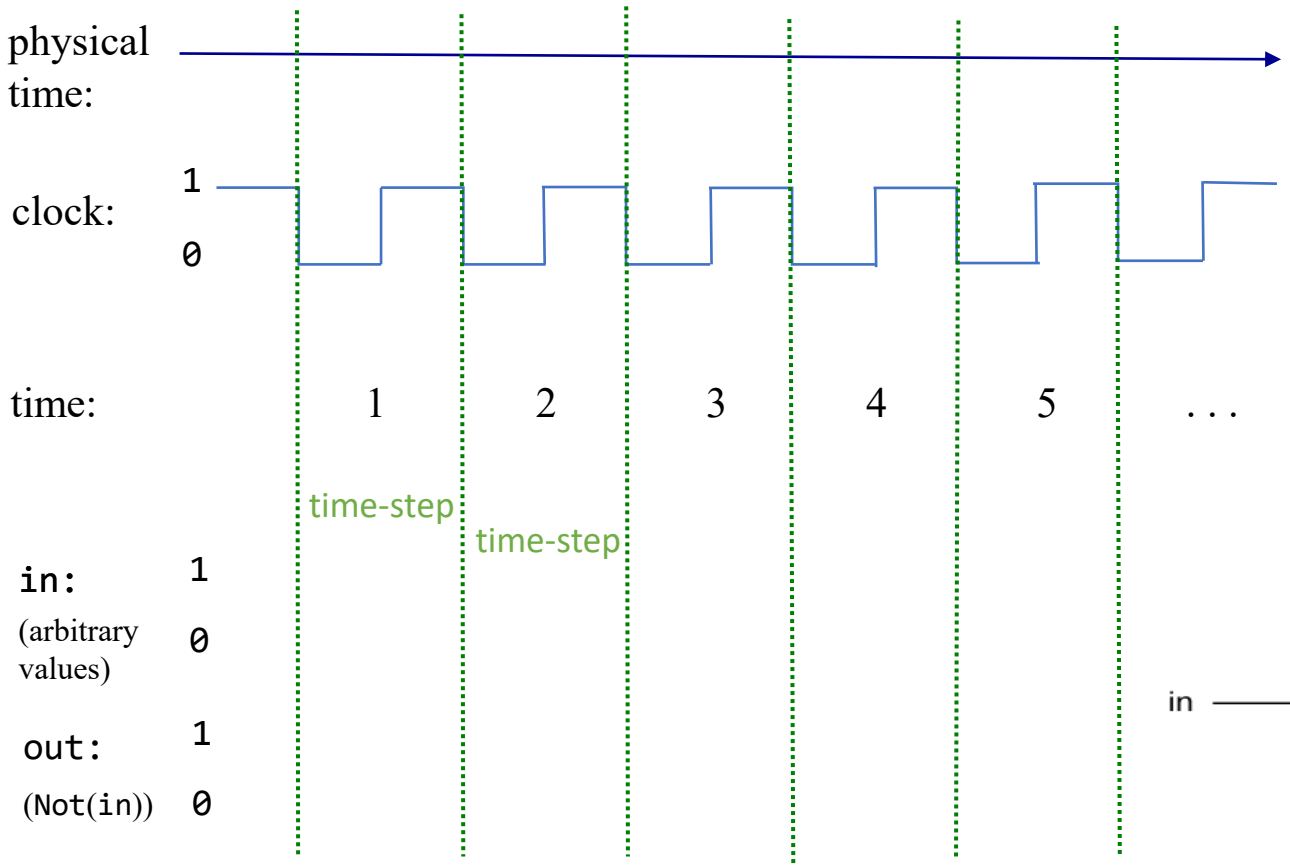
连续的时间

Discrete time:

Design decision:
Track state changes only when advancing from one time-step to another

离散的时间
设计决策：仅当一个时间步长前进到另一个时间步长时，轨迹状态才会改变

Chip behavior over time (example: Not gate)



Arrow of time:

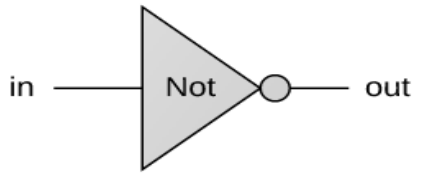
Continuous

连续的时间

Discrete time:

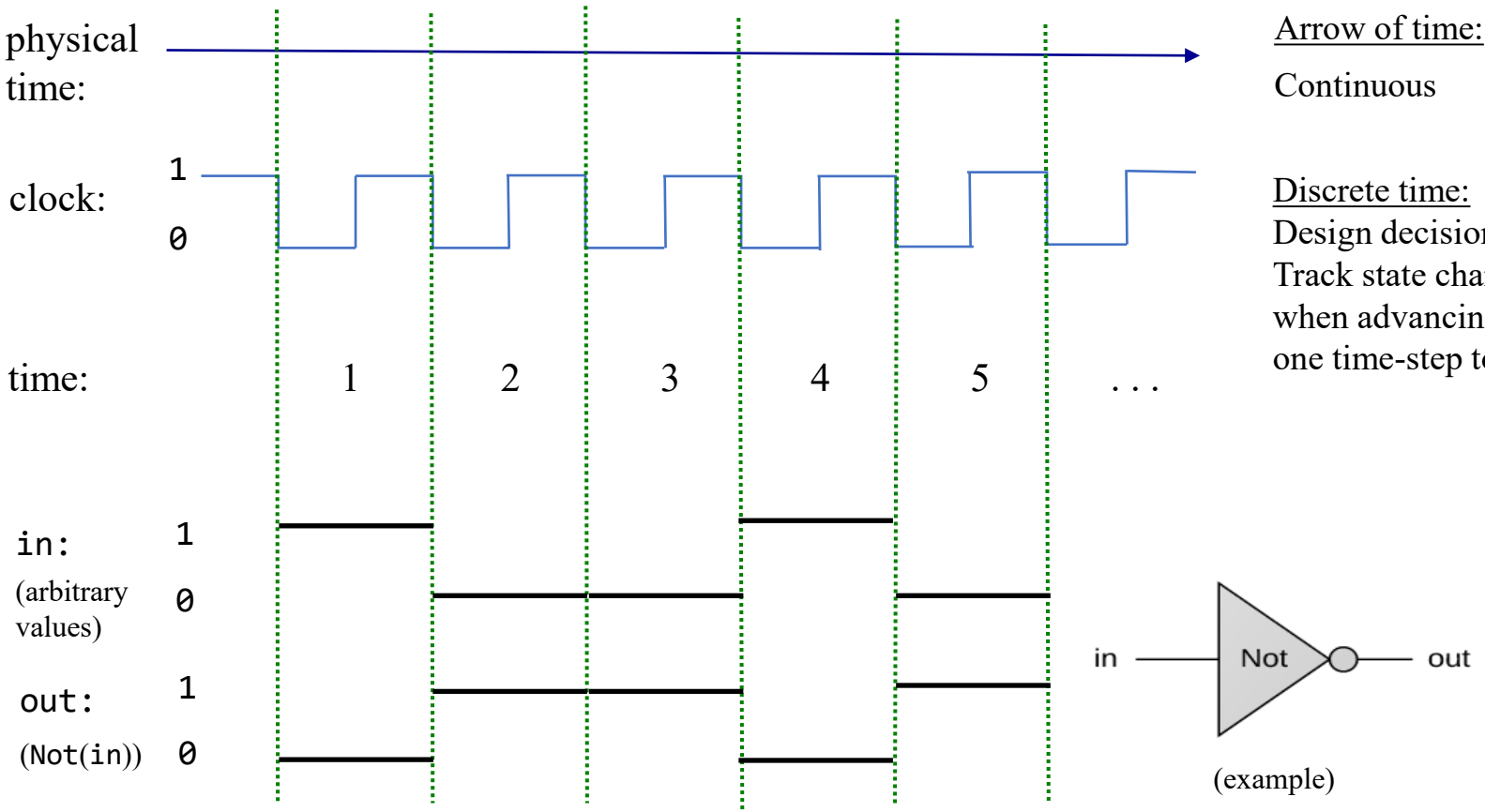
Design decision:
Track state changes only when advancing from one time-step to another

离散的时间
设计决策：仅当一个时间步长前进到另一个时间步长时，轨迹状态才会改变



(example)

Chip behavior over time (芯片随时间变化的行为 example: Not gate)



Arrow of time:

Continuous

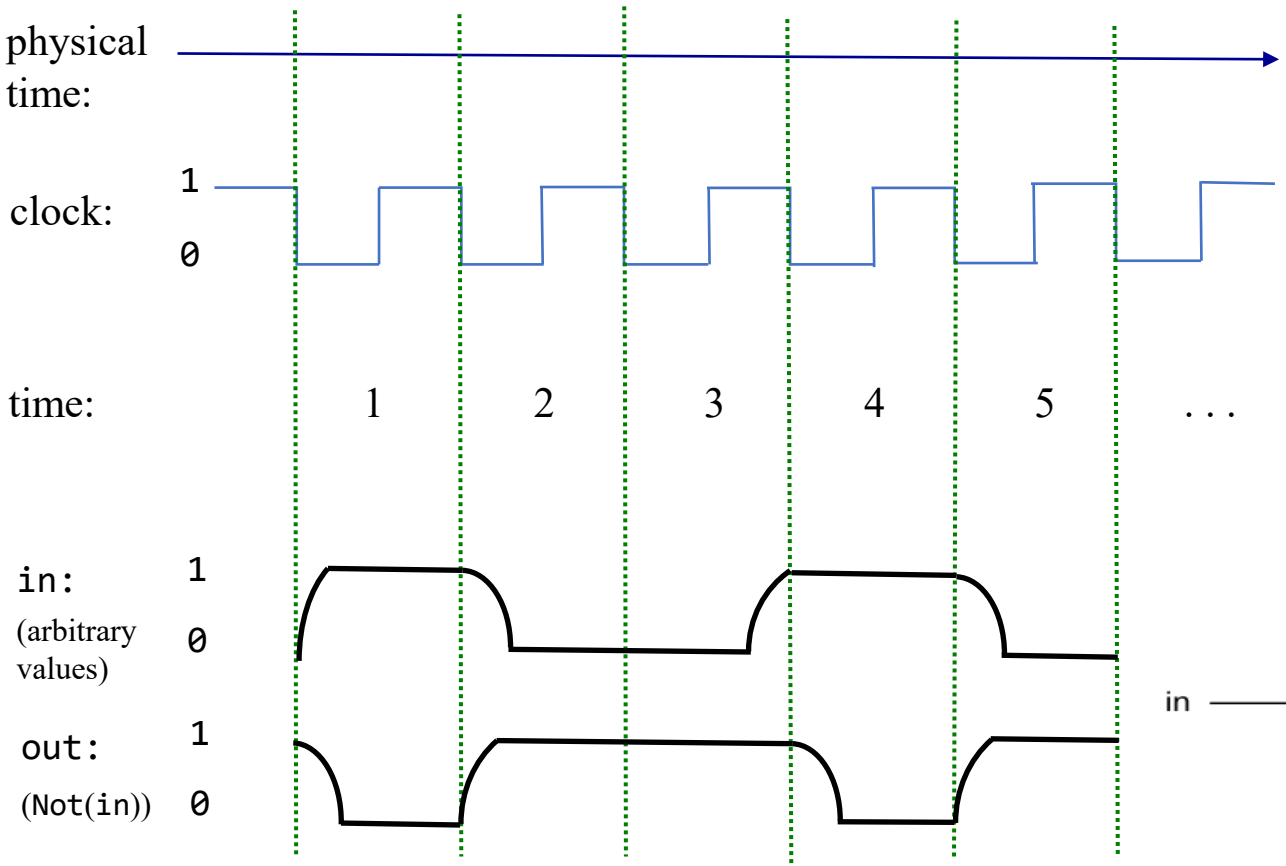
Discrete time:
Design decision:
Track state changes only when advancing from one time-step to another

(example)

输入和输出信号的期望/理想化行为:

这就是我们希望硬件的行为方式

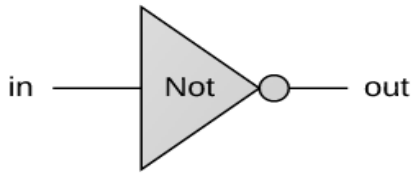
Chip behavior over time (example: Not gate)



Arrow of time:

Continuous

Discrete time:
Design decision:
Track state changes only when advancing from one time-step to another



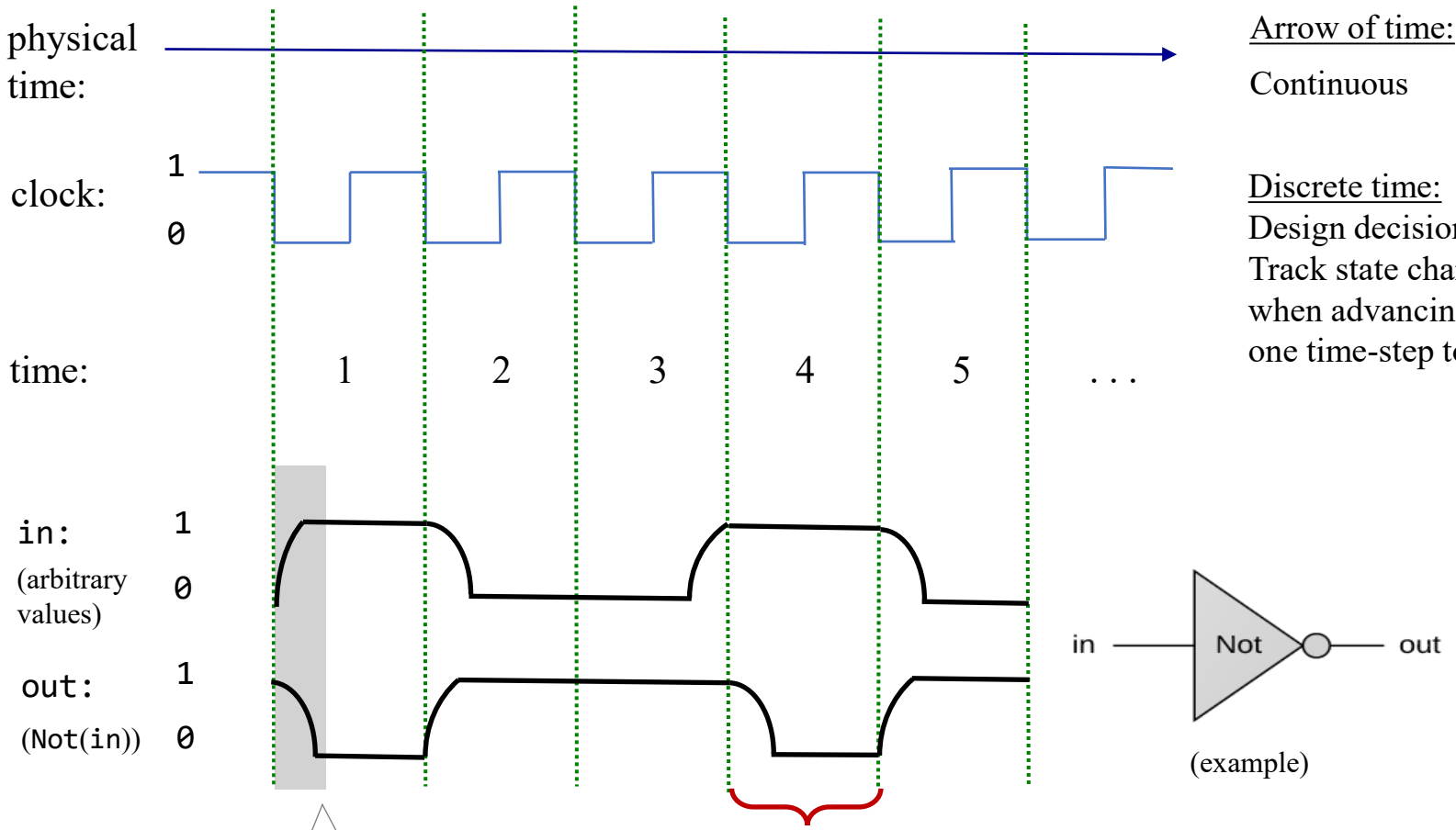
(example)

Actual behavior of the `in` and `out` signals:

Influenced by physical time delays

受物理时间延迟的影响

Chip behavior over time (example: Not gate)



Arrow of time:

Continuous

Discrete time:
Design decision:
Track state changes only when advancing from one time-step to another

(example)

Time delays
- Propagation delays
- Computation delays

Cycle length

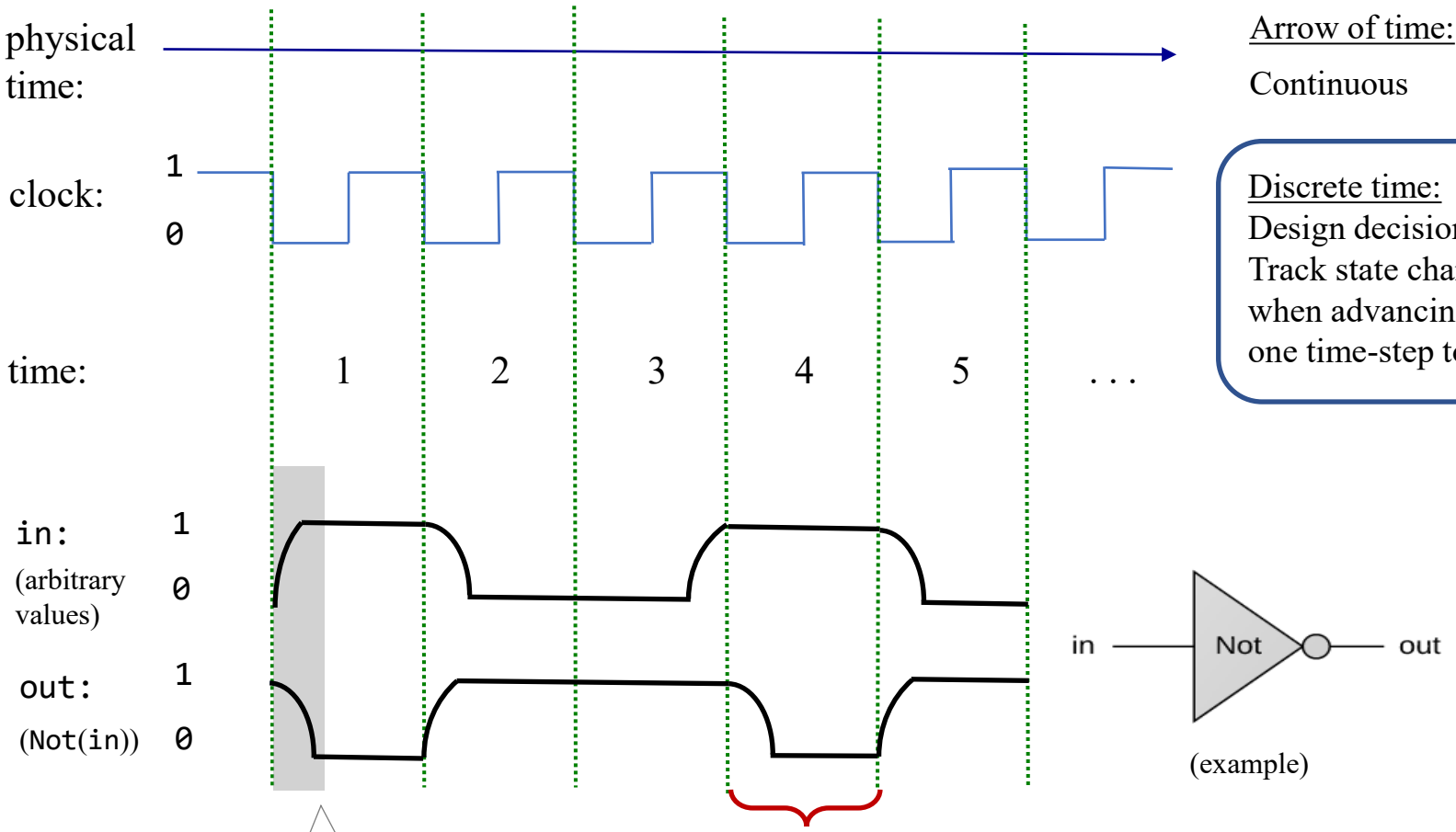
1. Propagation delays 传播延迟:

- chip中组合逻辑门的级联延迟累加，input信号需要按顺序依次通过这些门（后续门依赖前面门的输出信号），信号传播需要时间。
- 晶体管开关延迟

2. Computation delays 计算延迟

- 完成一次完整计算操作所需的总时间，包含传播延迟及其他系统开销
- 时钟周期约束
- 寄存器开销

Chip behavior over time (example: Not gate)



Arrow of time:

Continuous

Discrete time:
Design decision:
Track state changes only when advancing from one time-step to another

(example)

Time delays
- Propagation delays
- Computation delays

Cycle length
- Design parameter
- Set to be slightly > max(time delays)

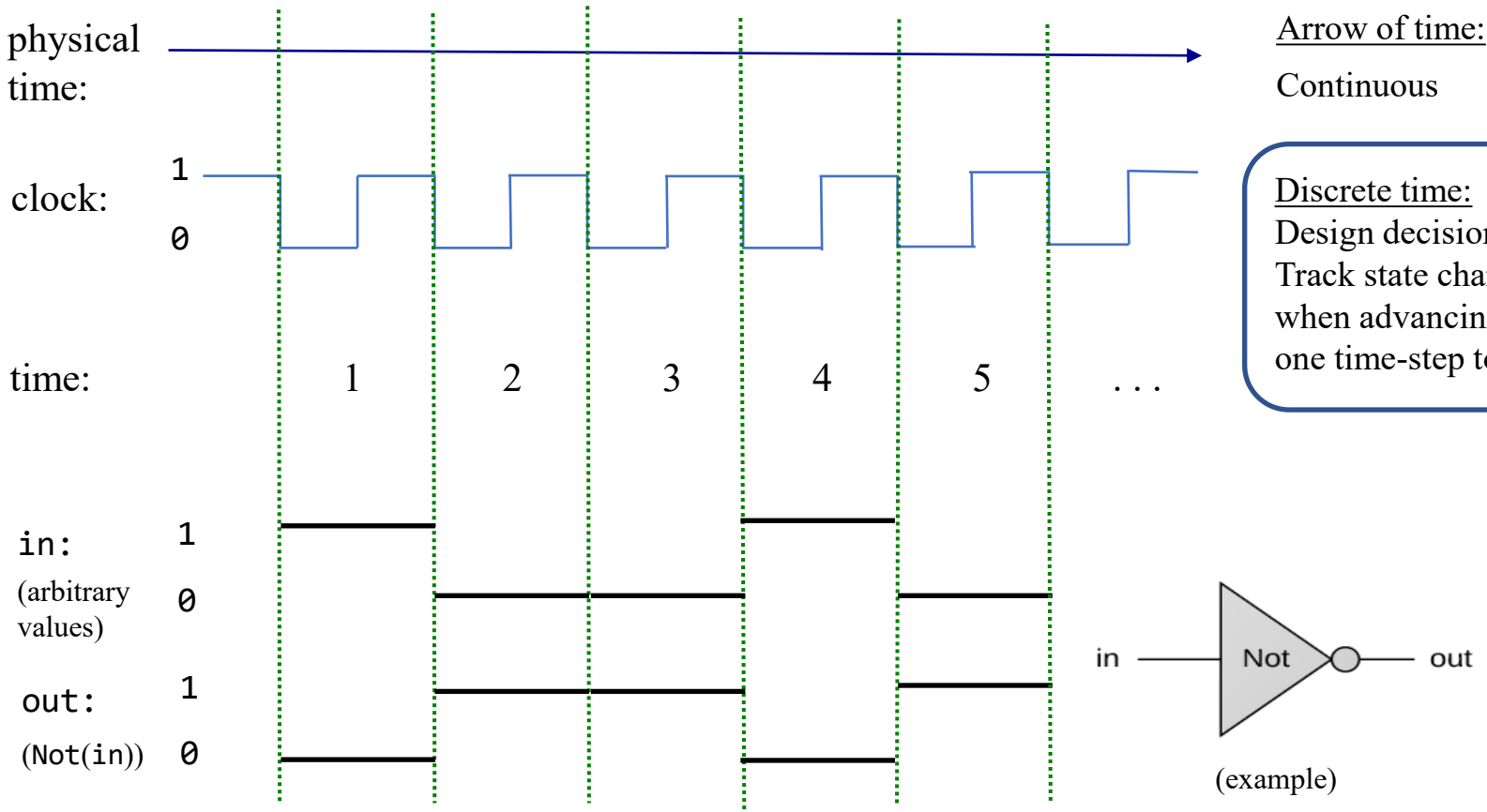
时钟周期长度设置为稍大于时间延迟的最大值

以 16 位加法器为例

传播延迟产生:
电流从 A[0] 输入 →
经过第一级全加器 →
进位传到第二级 → ...
最终和输出

计算延迟产生:
加法运算本身:
传播延迟（约 50ps）+
建立时间（20ps）+
时钟周期（200ps）

Chip behavior over time (example: Not gate)



Arrow of time:

Continuous

Discrete time:
Design decision:
Track state changes only when advancing from one time-step to another

(example)

Resulting effect:

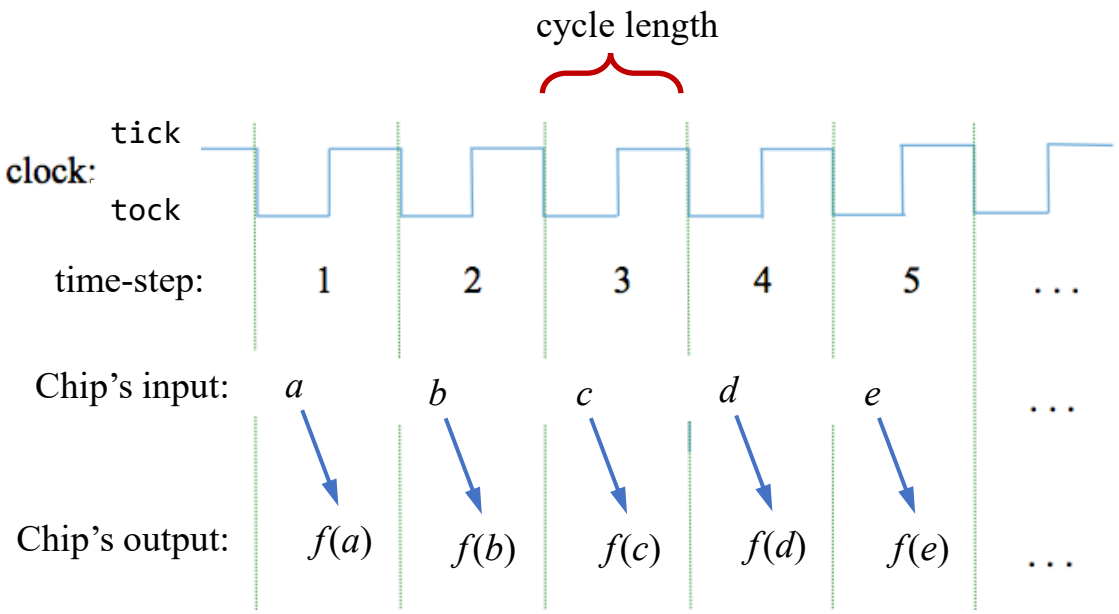
结果影响

- Combinational chips react “immediately” to their inputs

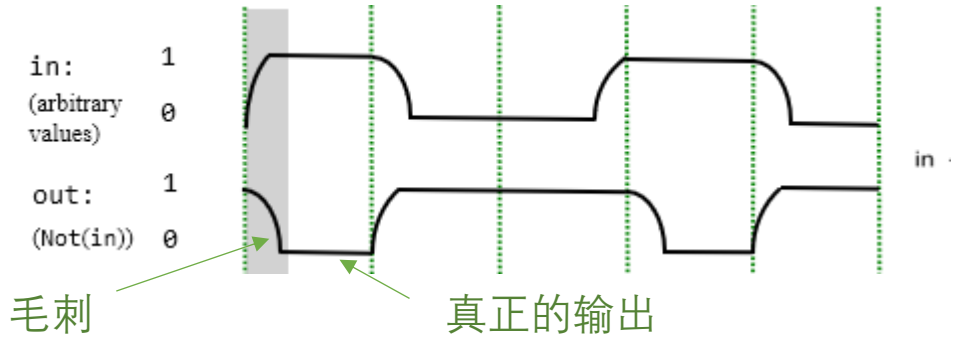
组合芯片对输入“立即”做出反应

- Facilitated by the decision to track changes only at cycle ends
  - 得益于「仅在时钟周期结束时采样状态变化」这一设计决策

The clock neutralizes the time delays if we decide to use *discrete time*



如果我们决定使用离散时间，时钟会抵消时间延迟



组合逻辑在输入变化时，输出可能短暂出现**尖峰脉冲（毛刺）**，然后才稳定到正确值。

时钟边沿只在**周期末尾**采样，此时组合逻辑早已稳定，毛刺已被过滤掉。

- Set the *cycle length* to be slightly > than the maximum time delay, and…

将时钟周期长度设置为略大于最大时间延迟

- Decide to use the chips's outputs only at the end of cycles (time-steps), ignoring what happens within cycles

只在周期结束时（时间步长）使用芯片的输出，忽略周期内发生的事情

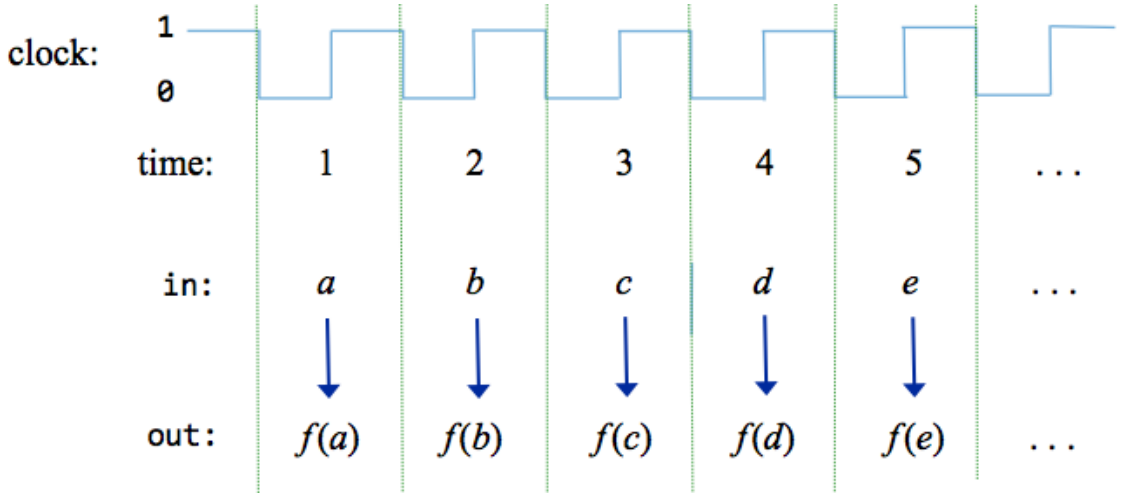
Combinational logic (组合逻辑电路)

Combinational logic:

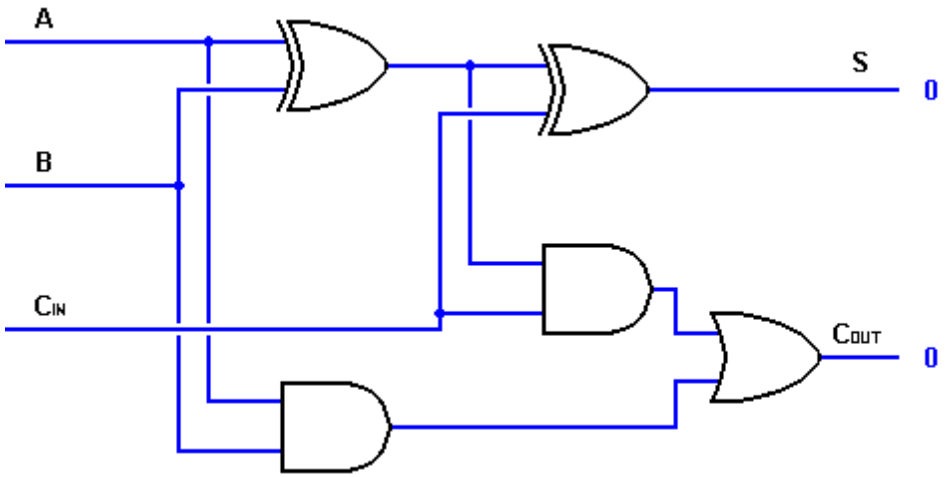
The output depends on the current inputs

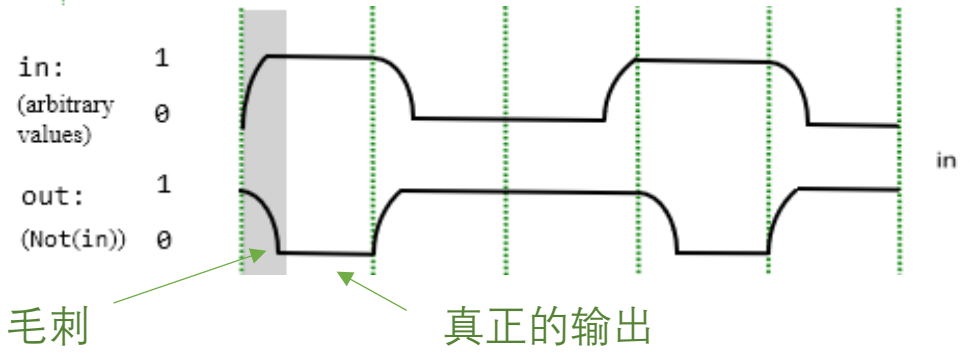
The clock is used to stabilize outputs

时钟用于稳定输出



Combinational logic的例子



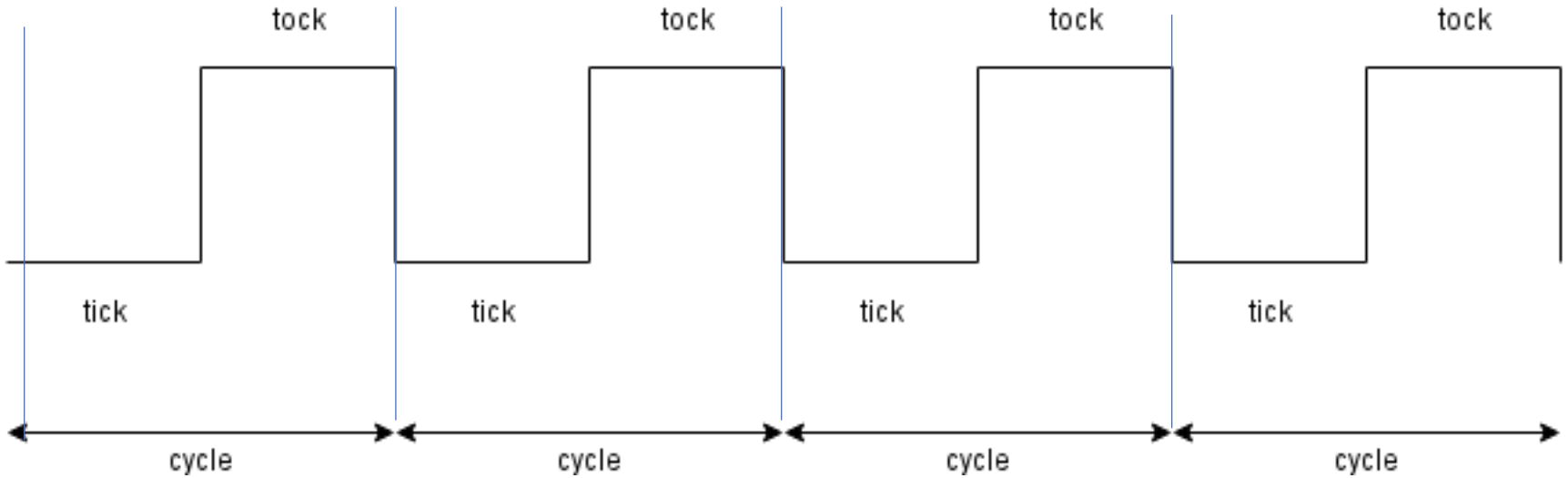


组合逻辑在输入变化时，输出可能短暂出现**尖峰脉冲（毛刺）**，然后才稳定到正确值。

时钟边沿只在**周期末尾**采样，此时组合逻辑早已稳定，毛刺已被过滤掉。

Clocks（时钟）

- In most computers the passage of time（时间流逝） is represented by a master clock（主时钟）
- A clock has two phases tick/tock (or low/high) （一个时钟有两个相位嘀/嗒，或高/低）
- The frequency of the clock, determine the speed of computation
- The waveform need not be regular it is the transition that is important(时钟波形无需规整，关键在于其电平跳变: low -> high or high -> low)



为什么需要Clocks（时钟)?

1. 解决"异步混乱"问题（Synchronization）

数字电路由**组合逻辑**（如加法器、ALU）和**存储元件**（触发器/寄存器，Flip-Flops）组成。

- **没有时钟时（异步电路）**：信号传播延迟不确定，一个门的输出变化快，另一个慢。当A门的输出作为B门的输入时，如果A变化太快，B可能捕获到**毛刺（Glitch）或中间态（Metastability）**，导致计算结果错误。
- **有时钟时**：所有寄存器只在**时钟边沿（Clock Edge）**同时采样输入。这相当于给整个系统一个统一的"快门时间"，确保所有数据在稳定后才被锁存。

> **类比**：就像交响乐团需要指挥（时钟），否则小提琴和鼓手各玩各的，声音就是噪音。

---

2. 过滤组合逻辑毛刺（Glitch Filtering）

组合逻辑（如进位链复杂的加法器）在输入变化时，输出可能短暂出现**尖峰脉冲（毛刺）**，然后才稳定到正确值。

- **异步设计**：毛刺会被直接送入下一级，导致错误计算。
- **同步设计**：时钟边沿只在**周期末尾**采样（如你之前问的 "track changes only at cycle ends"），此时组合逻辑早已稳定，毛刺已被过滤掉。

---

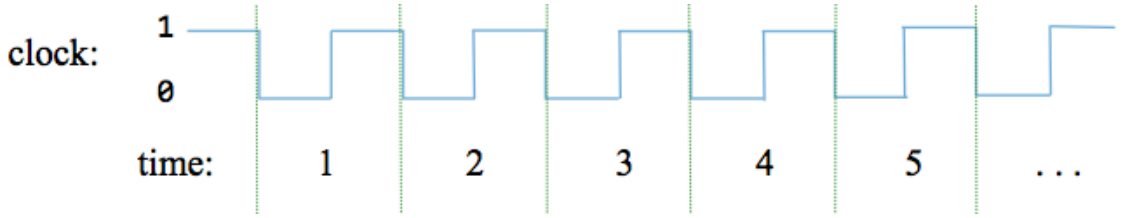
4. 时序分析与可制造性（Timing Analysis）

时钟将电路划分为离散的**时间片（Clock Cycle）**：

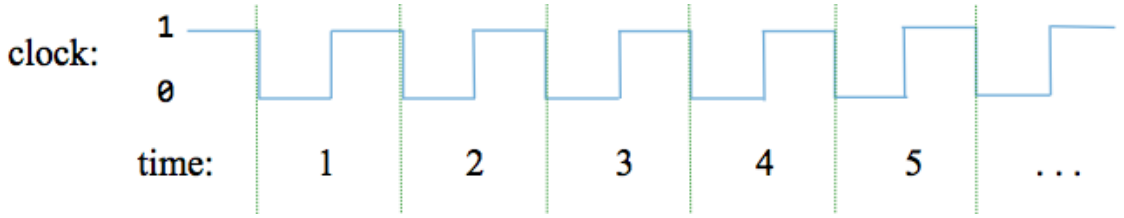
- **关键路径（Critical Path）**：每个周期内，信号必须走完最长的逻辑路径（如乘法器、复杂ALU）。
- **时钟周期约束**：只要保证 $T_{clock} \geq$ 关键路径延迟 + 建立时间，电路就能可靠工作。

这使得芯片设计可以进行**静态时序分析（STA）**，确保在不同工艺角（Process Corners）、温度和电压下都能工作。而异步电路无法进行这种全局时序约束。

Clock



Clock: Simulated implementation(时钟：模拟实现)

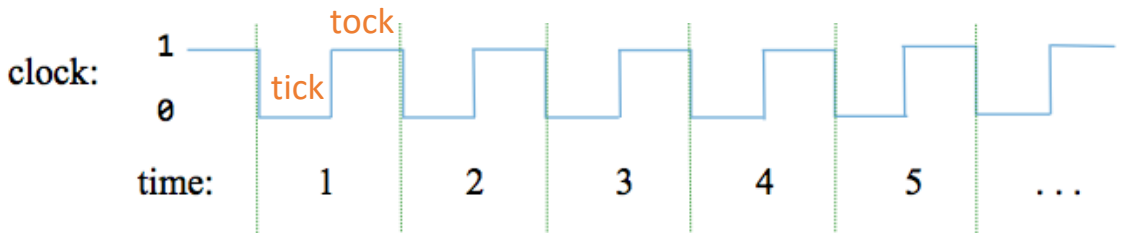


Interactive simulation(交互模拟)

A clock button can be used to generate a sequence of tick-tock signals:

`0, 0+, 1, 1+, 2, 2+, 3, 3+, ...`

Clock: Simulated implementation



Interactive simulation (交互模拟)

A clock icon can be used to generate a sequence of tick-tock signals:

`0, 0+, 1, 1+, 2, 2+, 3, 3+, ...`

Script-based simulation(基于脚本的模拟)

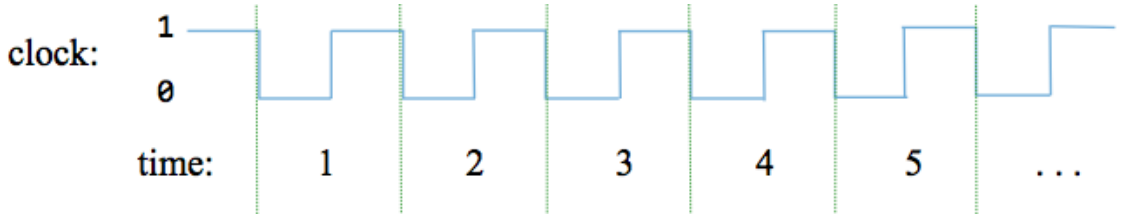
"`tick`" and "`tock`" commands can be used to advance the clock:

```
...
// Sets inputs, advances the clock, and
// writes output values as it goes along.
set in 19,
set load 1,
tick,
output,
tock,
output,
tick, tock,
output,
...
```

"tick"和"tock"命令可用于前进一个时钟周期:

- tick对应低电平 （逻辑0）
- tock对应高电平（逻辑1）
- 执行tock的瞬间就是时钟上升沿，会触发所有时序器件（DFF、Bit、寄存器等）采样输入、更新输出。

Clock: Physical implementation (时钟：物理实现)



Physical clock

- An *oscillator* is used to deliver an ongoing train of "tick/tock" signals
- 振荡器用于传递一系列持续的"滴答/滴答"信号

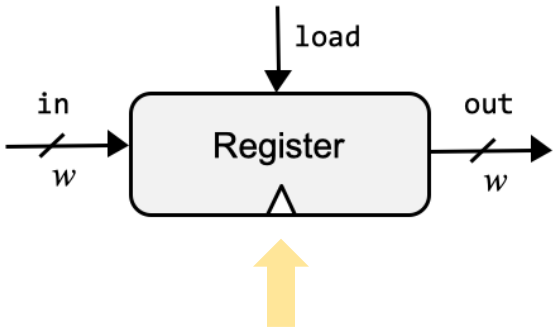


"1 MHz electronic oscillator circuit which uses the resonant properties of an internal quartz crystal to control the frequency. Provides the clock signal for digital devices such as computers." (Wikipedia)

1 MHz电子振荡器电路，利用内部石英晶体的谐振特性来控制频率。为计算机等数字设备提供时钟信号

- The oscillator's output is connected to all the time-based (clocked) chips in the computer

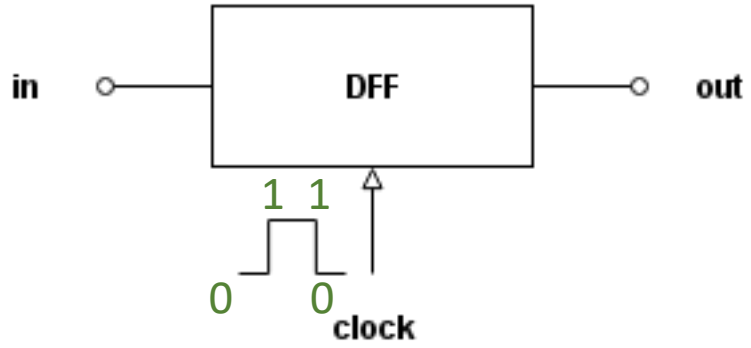
振荡器的输出连接到计算机中所有基于时间（时钟）的芯片



Chip diagram convention:

A triangle icon represents a clock signal input

Memory specification



- We want a logic circuit such that

  out(t) = in(t-1)

  where t is a discrete time interval

- The output of the gate at time t is dependent on the input of the gate at the previous time step (t-1)

  时间t的门输出取决于前一时间步（t-1）的门输入

- Functionality implemented by a Data Flip Flop (DFF)

  通过数据触发器（DFF）实现的功能

out(t) = in(t-1)

t	in	out
0	0	?
1	1	0
2	1	1
3	0	1
4	1	0

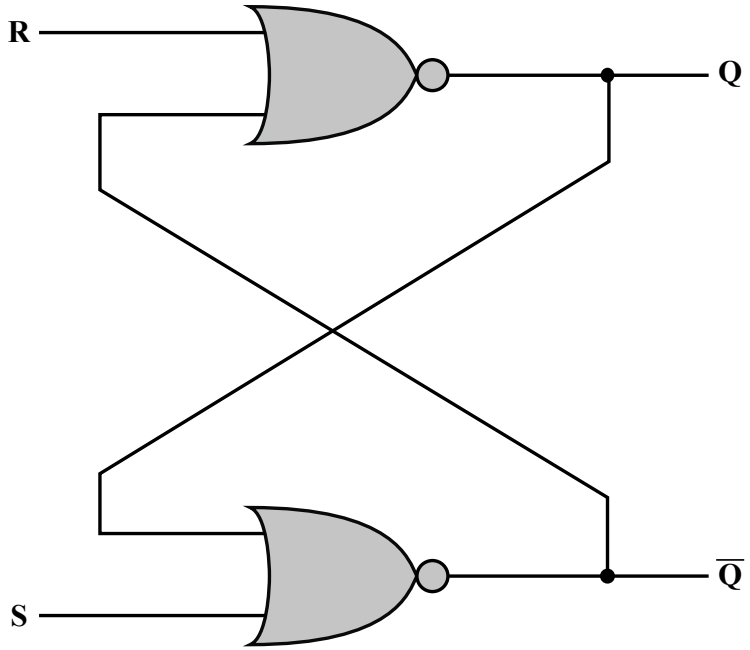
S-R Latch（异步S-R锁存器）

- Simple flip flop（简单翻转）
- S bit is set bit
- R bit is reset bit
- Implemented as 2 NOR gates

NOR = not (a or b)

- S and R set bit stored as 1 or 0 respectively

S和R设置位分别存储为1或0

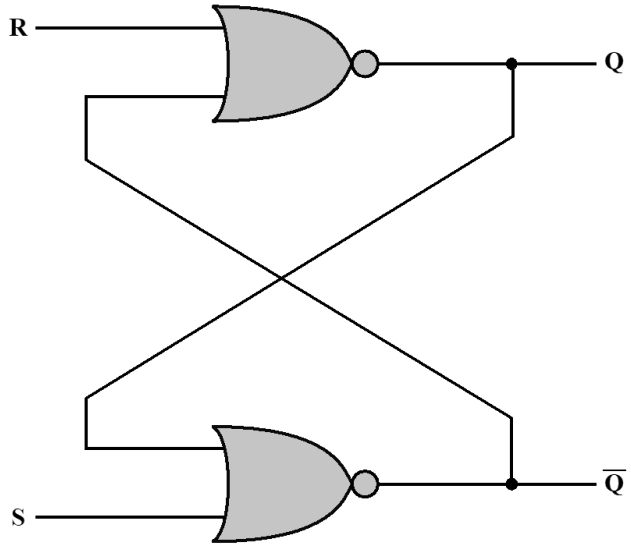


S-R锁存器是数字电路里**最基础的异步1比特存储单元**，所有更复杂的寄存器、内存都是由它衍生而来。
它是第一个能「记住1比特数据」的逻辑单元，有两个输入、两个互补输出：

- 当S=1，R = 0 时，Set 1Bit = 1
- 当S=0，R = 1 时，Reset 1Bit = 0
- 当S=0，R = 0 时，1Bit内容保持不变。实现最简单的内存「记忆」功能。
- 当S=1，R = 1 时，电路会产生随机值，实际电路中永远不会出现这个组合。

Characteristic Table

- Similar to truth table
- Determines next state based on inputs and current state
- 所谓current state, 就是存储在S-R Latch中的1-bit数
- 所谓next state，就是Q反馈回input时算出来的下一个output

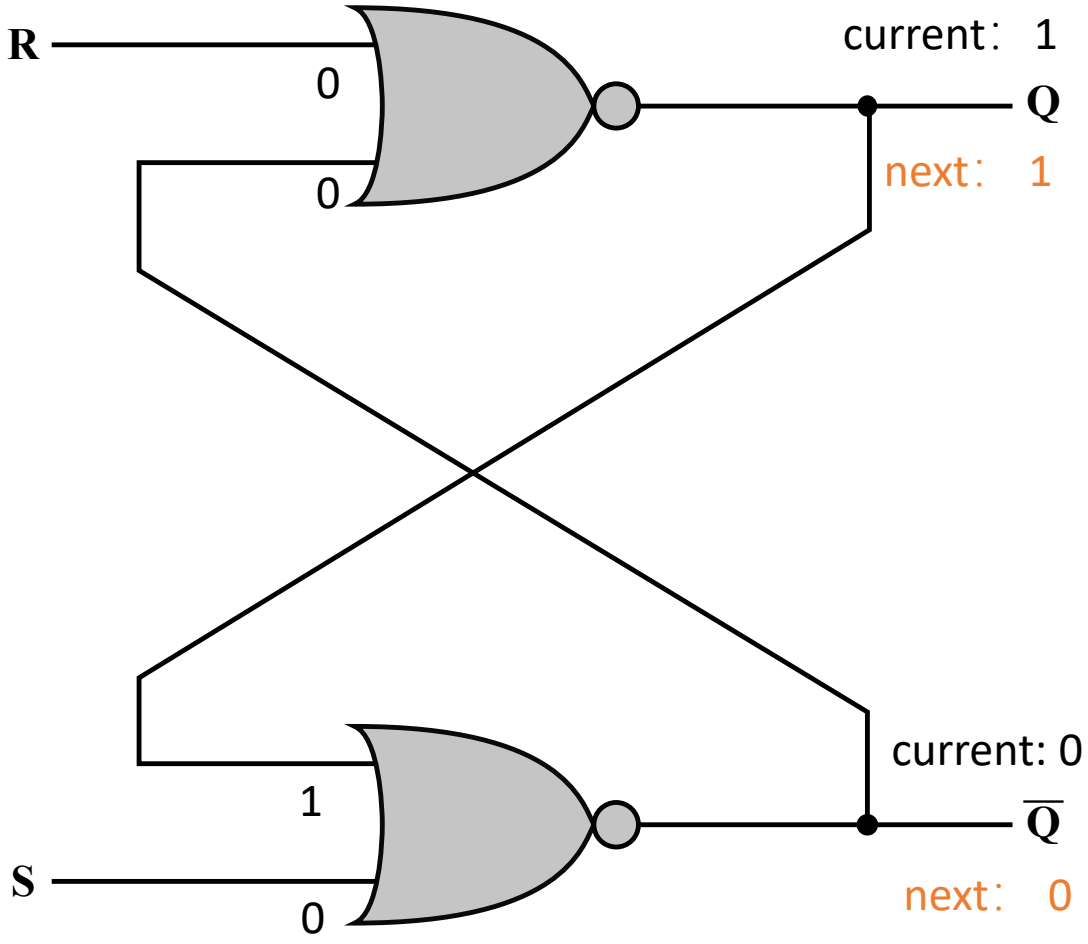


Simplified Characteristic Table			
S	R	Q_{n+1}	State
0	0	Q_n	Hold
0	1	0	Reset
1	0	1	Set
1	1	–	Invalid

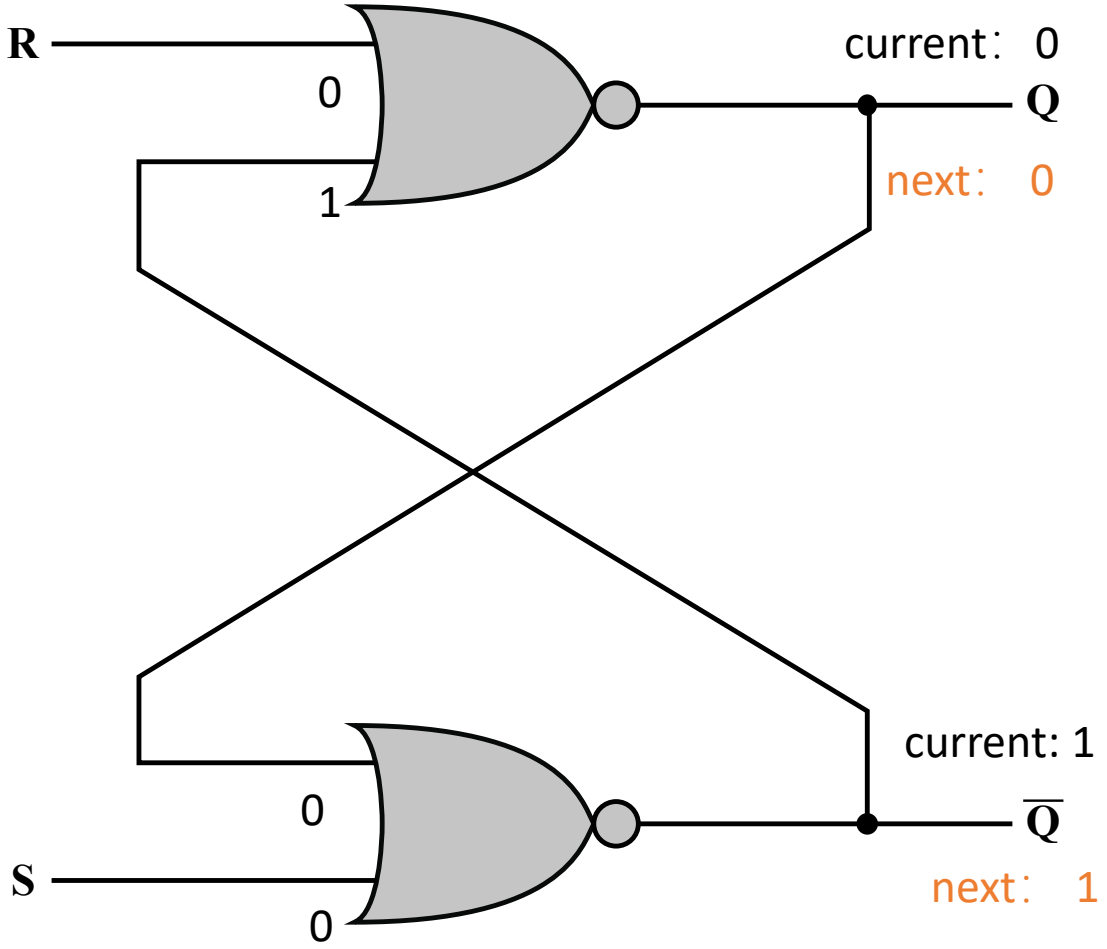
S	R	Current State	Next State	State
0	0	0	0	Hold
0	0	1	1	Hold
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	-	Invalid
1	1	1	-	Invalid

S-R Latch为什么能锁存住值（When S = 0 R = 0）

情景1： Current State Q = 1



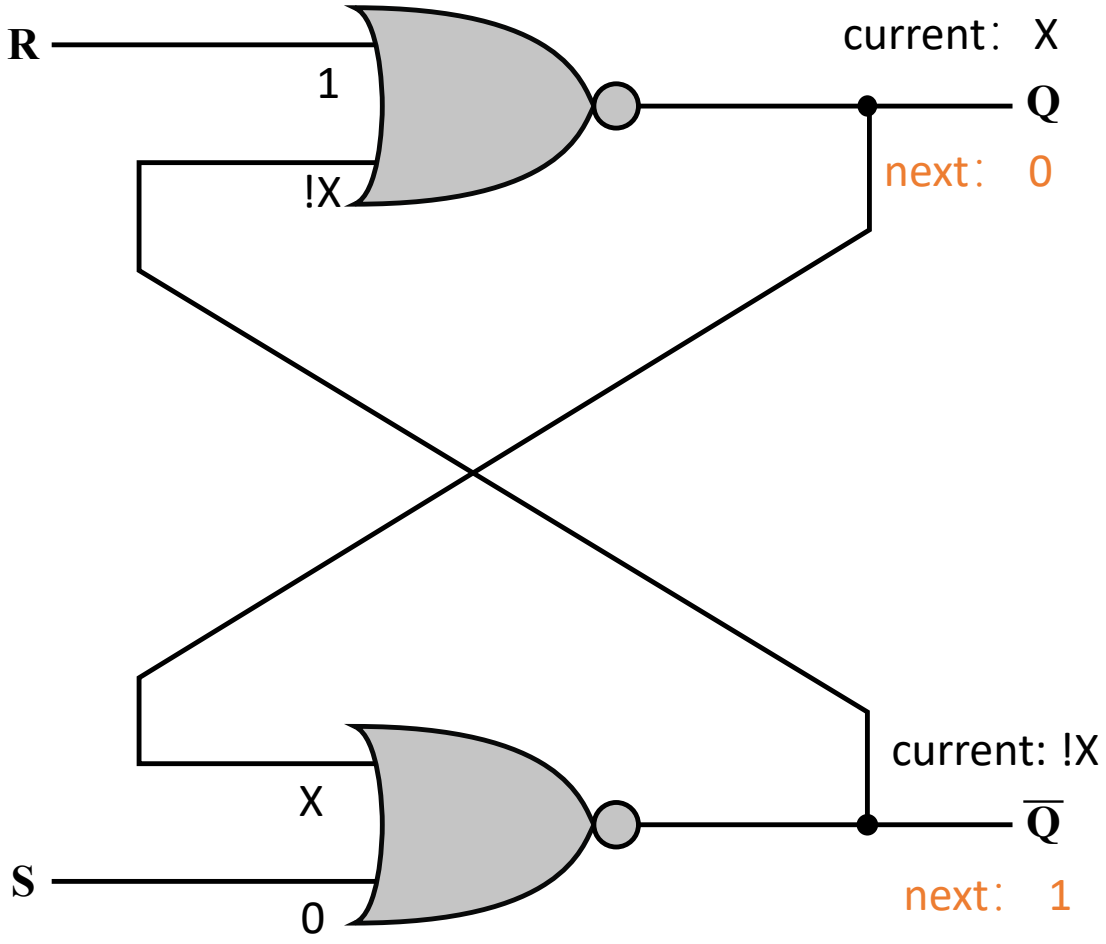
情景2： Current State Q = 0



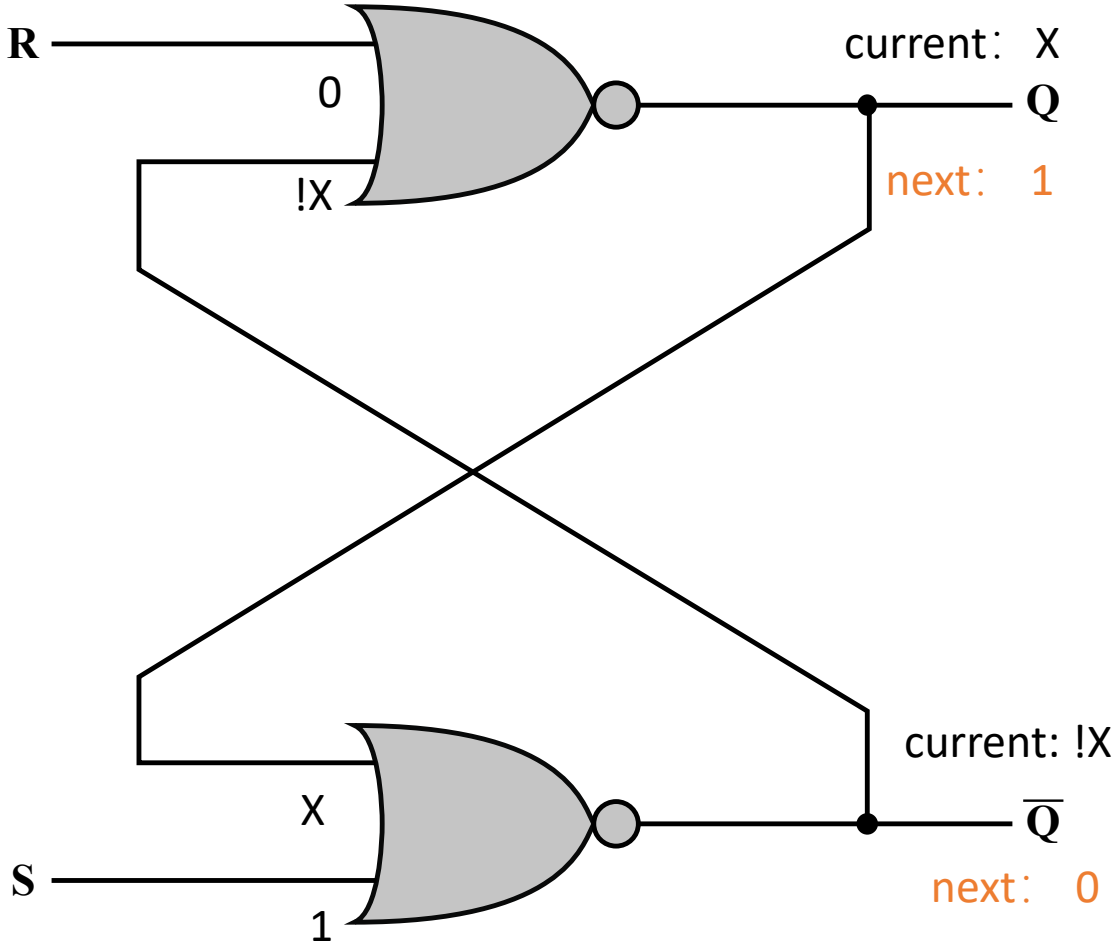
由此可见： 当 S = 0， R = 0时， Q内无论存储是1还是0，Next State Q都保持不变，即内存的值锁存住了

S-R Latch修改值(内存写)

情景1： S = 0 R = 1 Q任意： Q清零



情景2： S = 1 R = 0 Q任意： Q置1



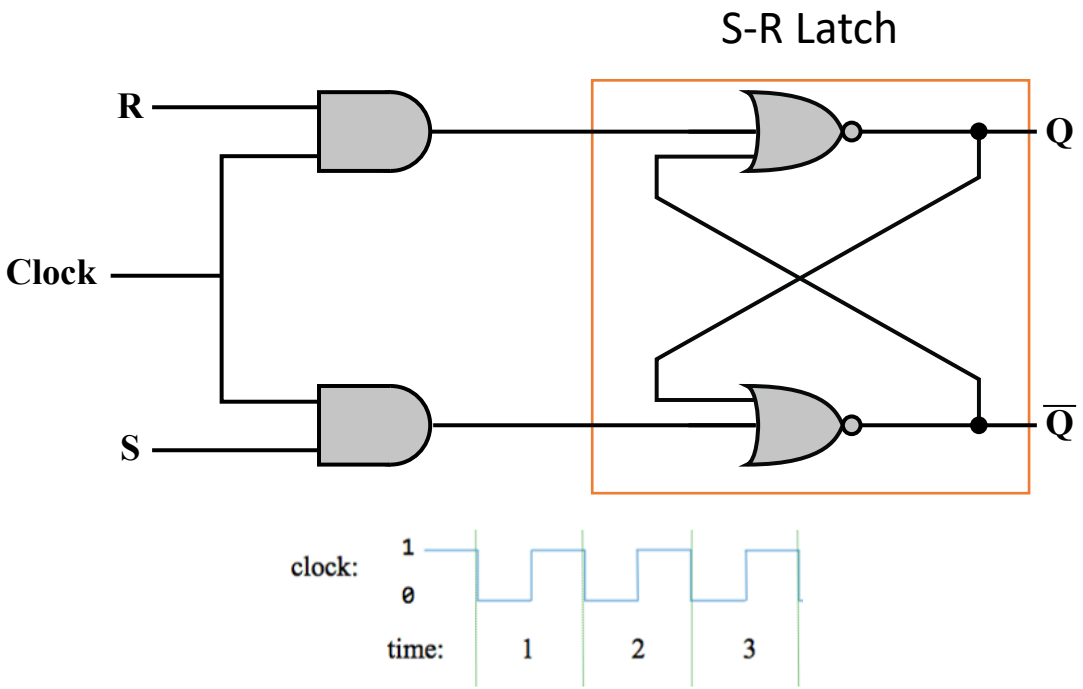
Clocked S-R Latch(同步时钟控制S-R锁存器，S-R Latch + Clock同步)

- S-R Latch is asynchronous(异步)
  - It will change state after a certain time, but that time is not fixed

    它会在一段时间后改变状态，但该时间不是固定的
- Useful to have system where we know where we can predict when state will change

  如一个系统，可以预测状态何时会改变，这很有用
- Clocked SR Latch synchronises (同步) to clock pulse(时钟脉冲)
  - S & R inputs are only passed to NOR gates when clock pulses

    S&R输入仅在时钟脉冲到达时传递到NOR门



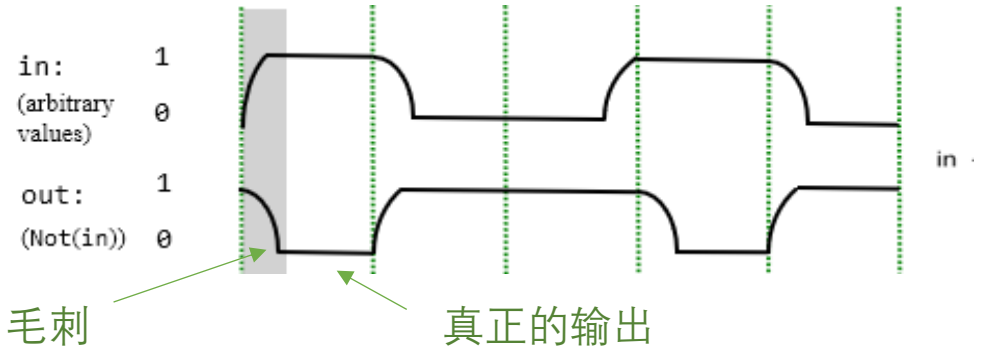
Clocked S-R Latch在S-R Latch前加入AND，用来控制R,S信号是否能进入S-R Latch。

- 当Clock低电平进入and门时，R,S信号被切断，此时R=0，S=0，Hold状态，数据被保持。
- 当Clock高电平进入and门时，R,S信号被放行，此时R=0，S=1 或 R=1 S=0；数据被置1或清0

用Clocked S-R Latch来改进S-R Latch的原因

S-R锁存器的两个大问题：

- 完全异步：S/R输入的任何跳变（哪怕是毛刺、干扰）都会立刻改变输出，没有统一的时间基准，大规模系统里会出现随机的逻辑冲突；
- 无控制窗口：输入永远生效，无法做到「只在需要的时候采样输入」。

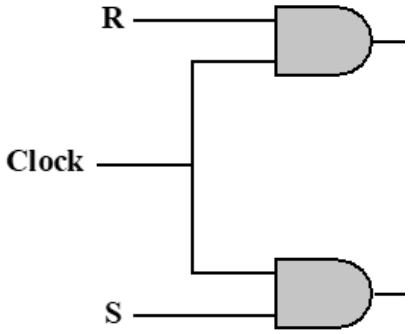


locked S-R Latch解决了这两个问题：

- 它加了时钟控制，只有时钟处于有效电平的窗口内，S/R的输入才会影响输出
- 时钟无效时，无论S/R怎么跳变，输出都保持原状态不变，完美实现了「同步屏蔽干扰」的需求。

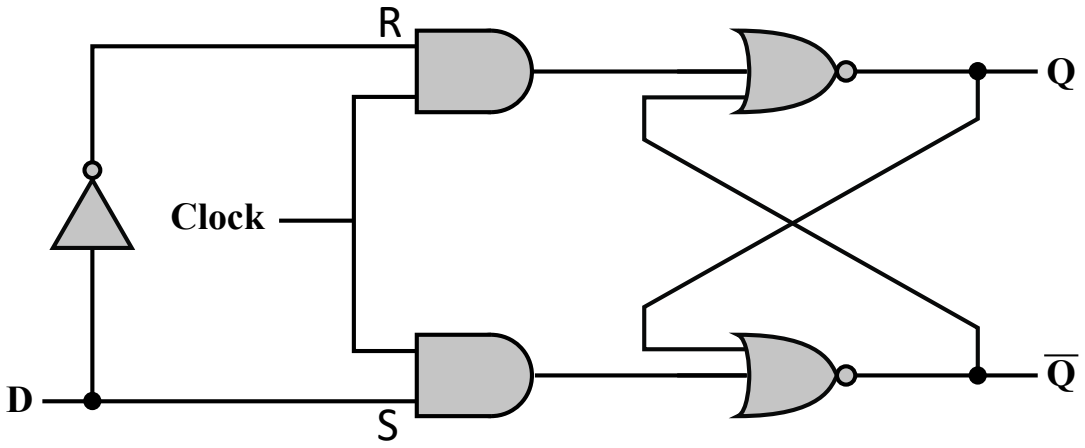
所有外部输入的信号（比如按键、传感器、其他模块的输出）都是异步的，自带毛刺、抖动，如果没有时钟控制，这些毛刺会直接触发普通S-R锁存器乱翻。加了时钟后，只有时钟有效窗口内的信号才会被采样，窗口外的干扰全部被屏蔽，输出稳定性大幅提升。

普通S-R锁存器的状态变化是随机的，发生在S/R跳变的任意时刻；而Clocked S-R Latch的状态变化只发生在CLK的有效窗口内，整个系统的所有状态变化都有统一的时间基准，方便做时序约束、时序分析，是构建复杂同步时序电路的基础单元



D Flip Flop（D触发器）

- Two sets of inputs which change state are S=0, R=1 and S=1, R=0
- Can be implemented as single input
  - One input goes to one NOR
  - Negated input to other NOR



Characteristic Table		
D	Q_{n+1}	State
0	0	Reset
1	1	Set

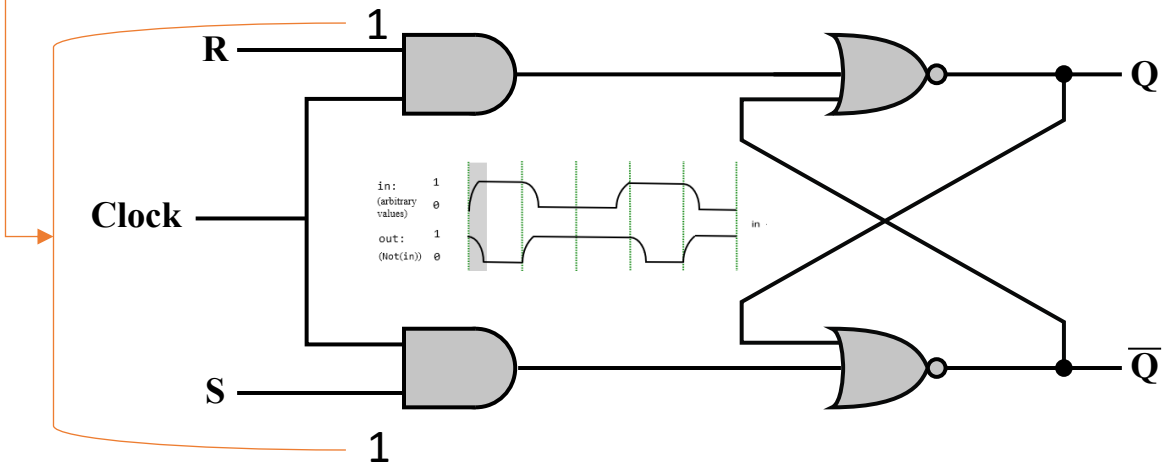
你可以把D触发器理解成一个只在闹钟响的瞬间记事的笔记本：

- 闹钟就是时钟（CLK），只有闹钟响（边沿到来）的瞬间，你才会记下当前看到的内容（D端的输入）；
- 闹钟不响的时候，不管周围发生什么（D端怎么变），你都一直记住上次记下的内容（输出Q保持）；
- 而且你只会记0或者1，不会出现矛盾的内容（没有禁止态），非常可靠。

用D Flip Flop来改进Clocked S-R Latch的原因

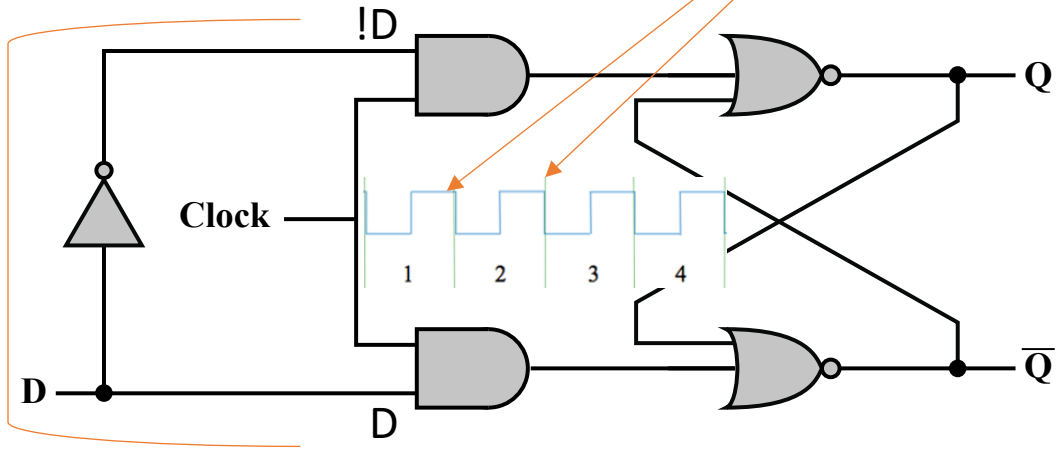
Clocked S-R Latch的2个问题：

- 依然存在禁止态： S和R同时为1时，输出Q和Q非都为0，违反逻辑互补规则，会导致电路冲突。
- 依然是电平敏感：CLK有效窗口内的毛刺、干扰还是会直接传到输出，抗干扰能力不如后面学的边沿触发器。



D Flip Flop解决了Clocked S-R Latch的2个问题：

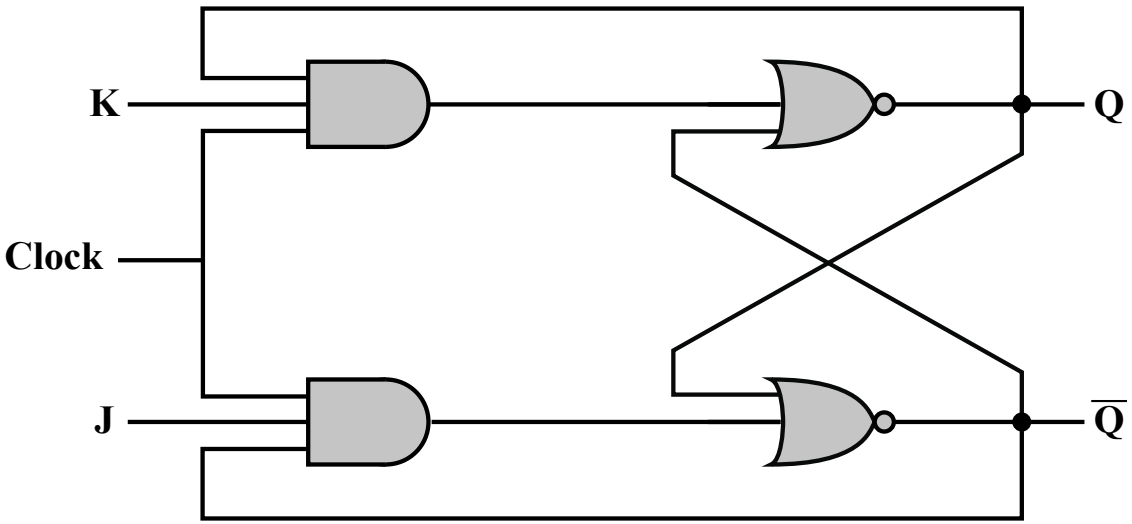
- 把Clocked S-R Latch 的S和R输入端用逻辑绑定，让S和R永远不可能同时为1，从根源上消灭禁止态；
- 通过结构设计，让它只在时钟**边沿（上升沿/下降沿）**到来时才更新输出，其他时间完全和输入隔离，彻底解决抗干扰问题。



Exercise

- Open https://logic.ly/demo/ and choose the D Latch (D flip flop)
- Toggle the input to see when the D Latch state changes.

J-K Flip Flop（J-K触发器）



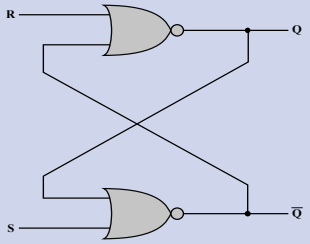
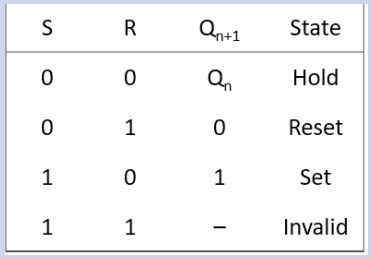
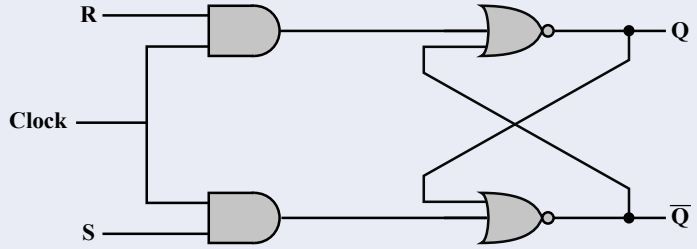
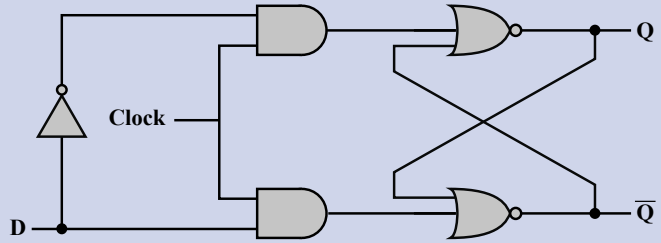
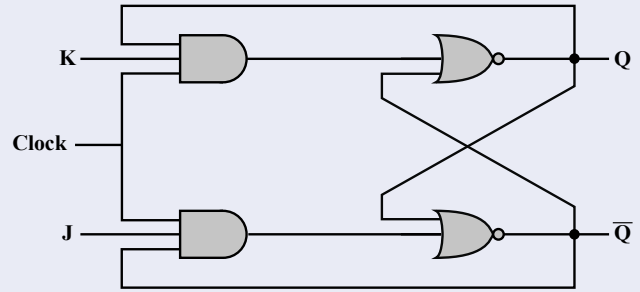
- All possible input values valid
- Same as S-R Flip Flop for inputs other than J=1 and K=1
- If J=1 and K=1 then the output is reverse of current state

JK触发器是基于cloked S-R latch进行改进的：

- JK触发器通过增加两个与门，直接把「S=R=1」的无效状态变成了**状态翻转**功能，完全消除了约束条件。
- 可以把它理解为一个「带锁的开关盒」：平时时钟没到的时候，不管J、K怎么调，盒子里的内容（Q的输出）都不会变；只有时钟这把「钥匙」到来时，才会根据提前设置的J、K值，决定盒子里放0、放1、保持原样，还是把内容反过来

Characteristic Table		
J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

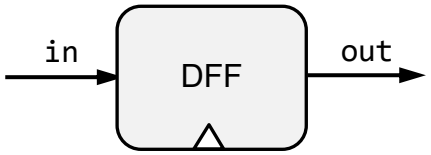
S-R Latch 、Clocked S-R Latch、D Flip Flop(DFF)、J-K Flip Flop对比

	电路图	特点	缺点
S-R Latch S-R锁存器			1. 抗干扰能力差（毛刺输入也反应） 2. 存在禁止态（S=1 R=1） 3. 状态变化是随机的，发生在S/R跳变的任意时刻
Clocked S-R Latch 时控S-R锁存器		• 加入时钟同步控制 • 状态变化发生在Clk来临 • **电平触发**（level-triggered）	1. 抗干扰能力比S-R Latch好，但CLK有效窗口内的毛刺、干扰还是会直接传到输出 2. 存在禁止态（S=1 R=1）
D Flip Flop(DFF) D触发器		• 加入时钟同步控制 • 消灭了禁止态 • 时钟**边缘触发**(edgetriggered)，抗干扰能力强	
J-K Flip Flop J-K触发器		• 加入时钟同步控制 • 将禁止态变成了状态翻转 • 时钟**边缘触发**(edgetriggered)，	

DFF （D Flip Flop）

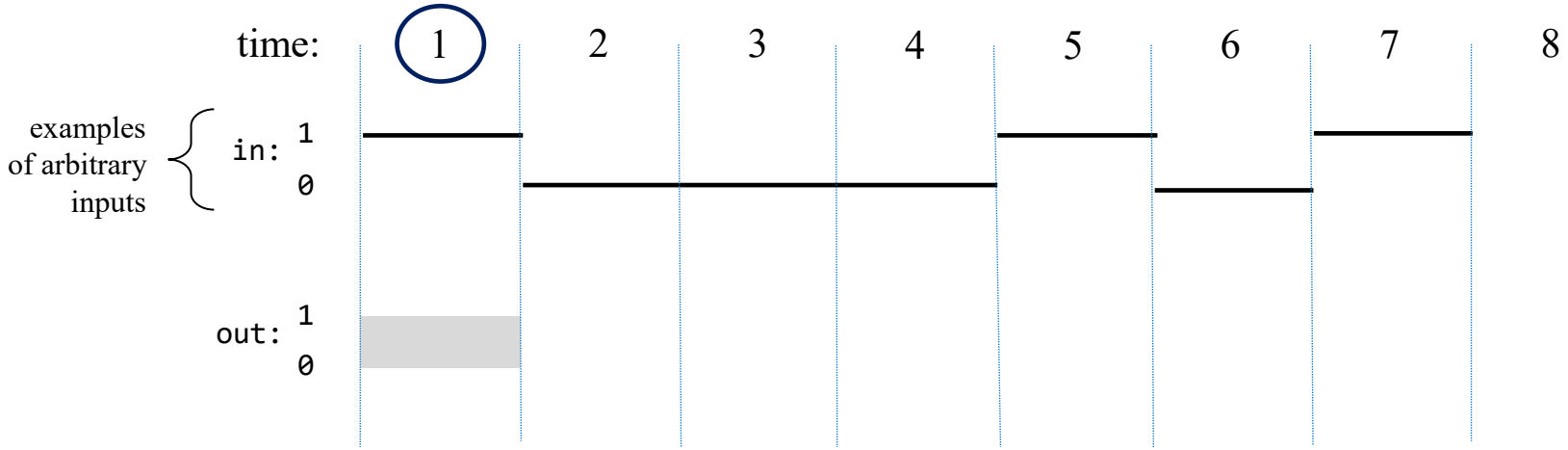
Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



$\texttt{out}(t) = \texttt{in}(t-1)$

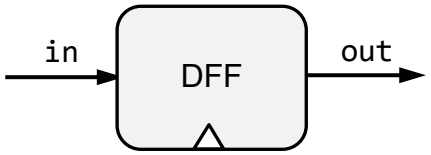
最基本的顺序门：输出前一时间步的输入



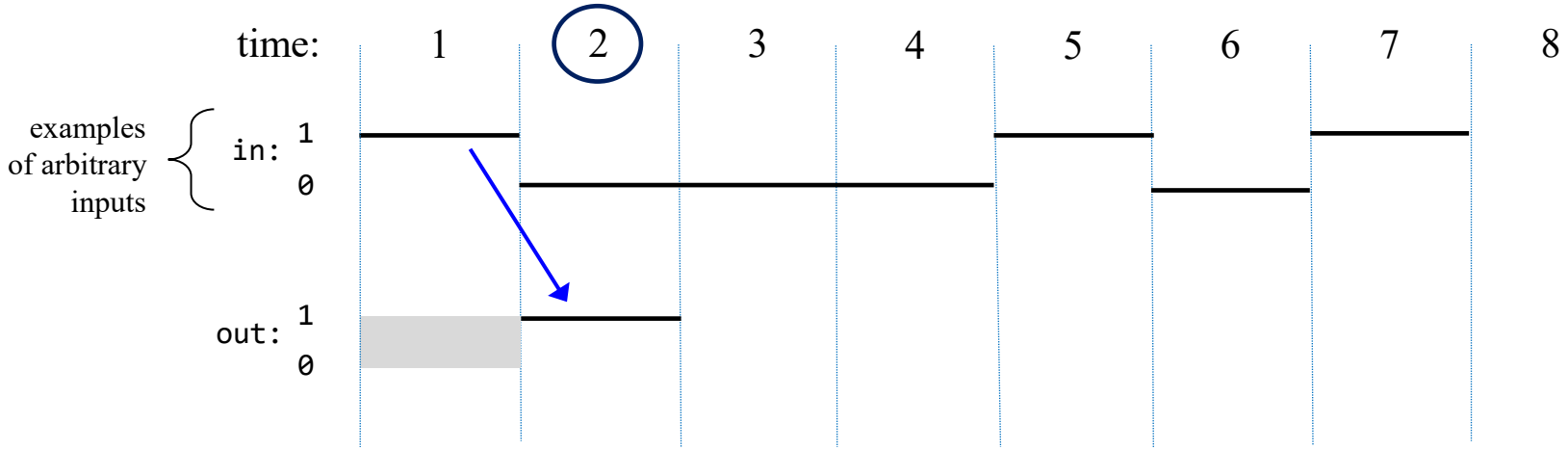
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



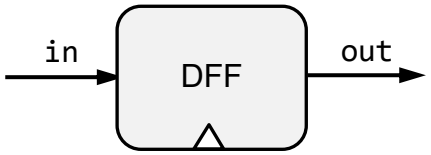
$\texttt{out}(t) = \texttt{in}(t-1)$



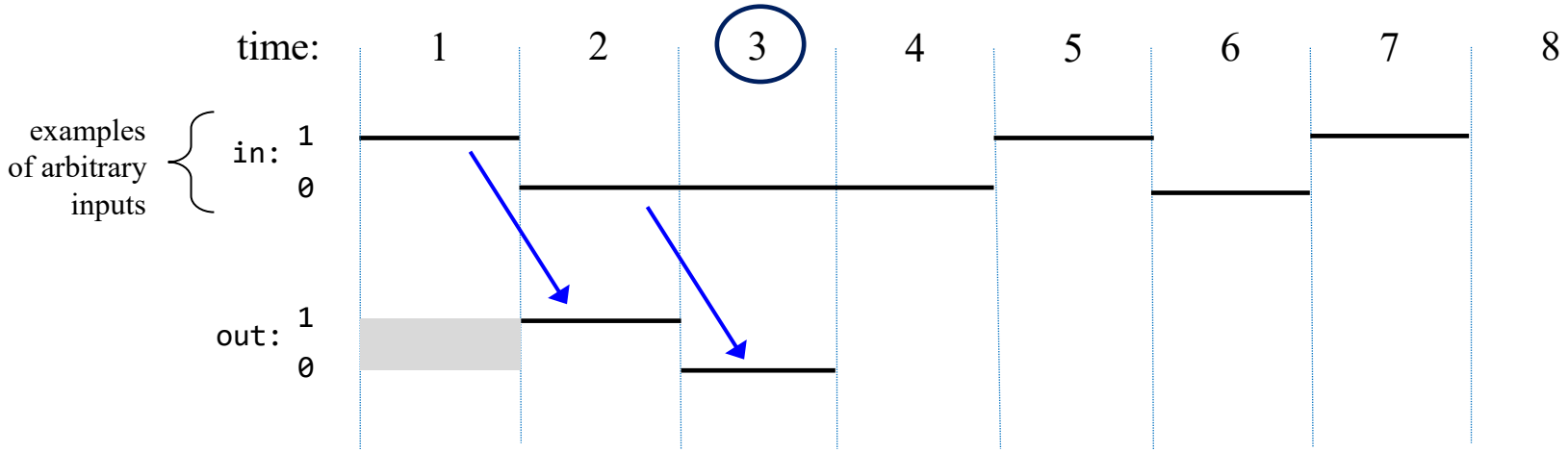
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



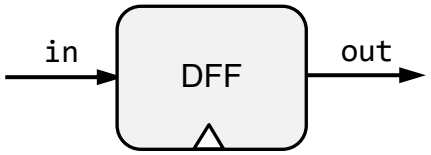
$\texttt{out}(t) = \texttt{in}(t-1)$



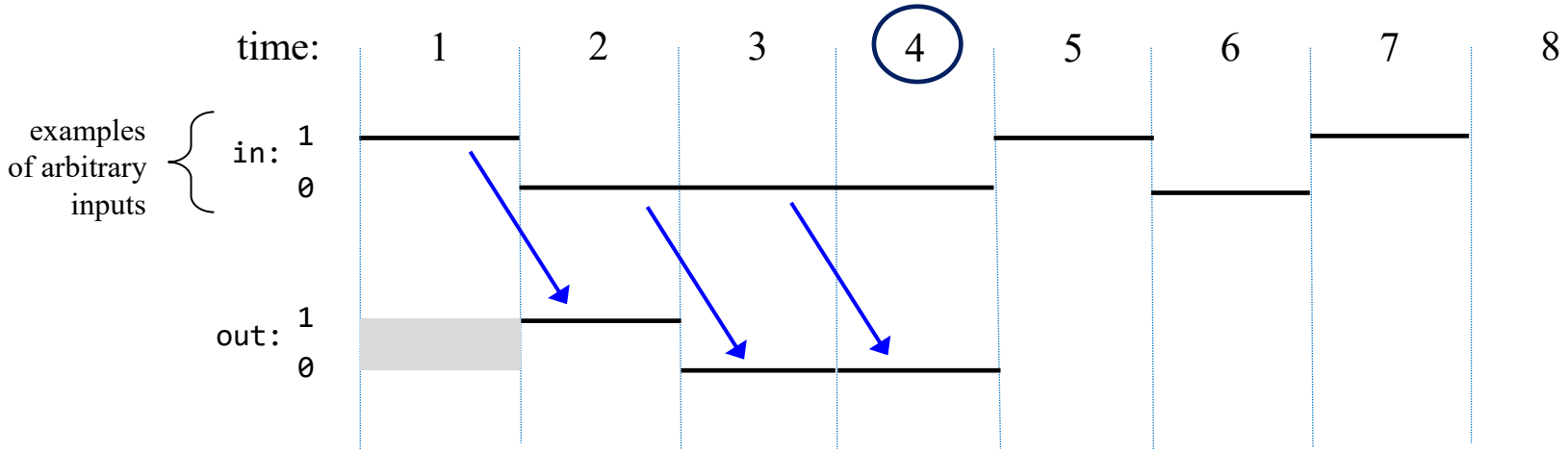
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



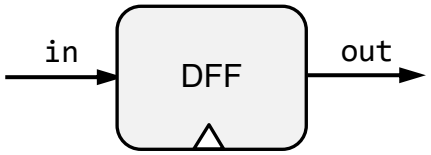
$\texttt{out}(t) = \texttt{in}(t-1)$



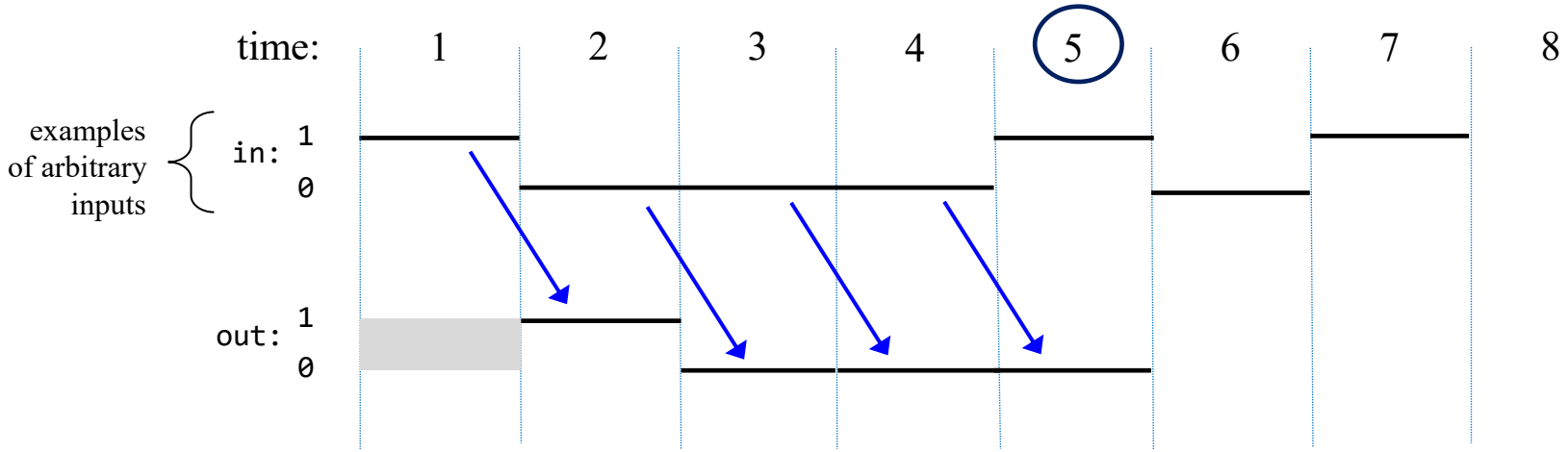
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



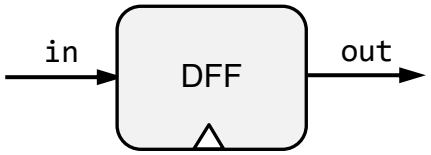
$\texttt{out}(t) = \texttt{in}(t-1)$



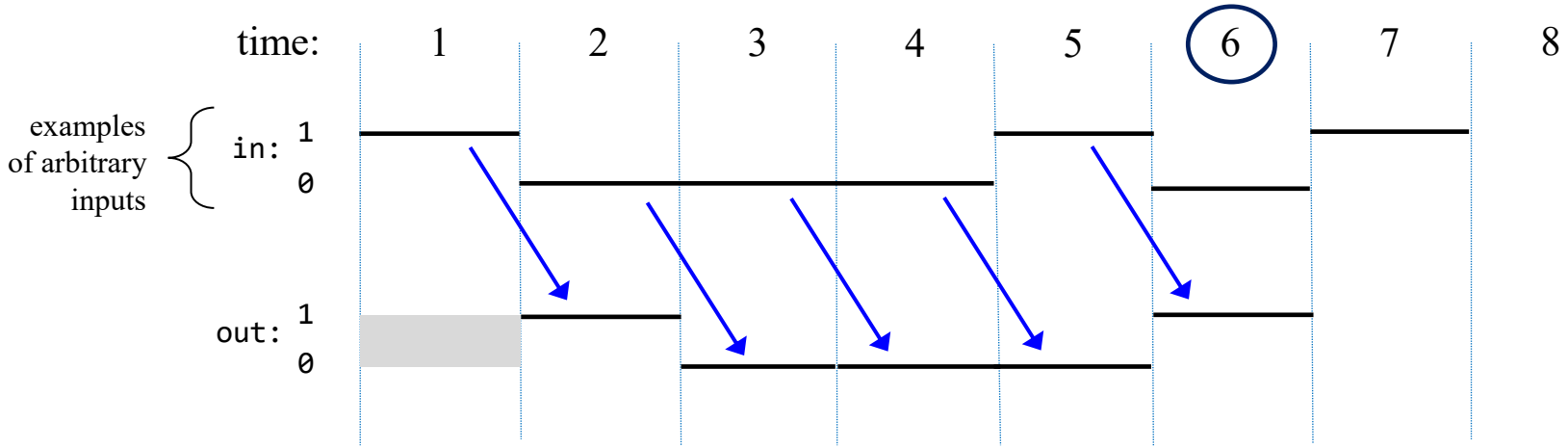
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



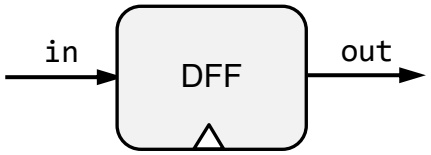
$\texttt{out}(t) = \texttt{in}(t-1)$



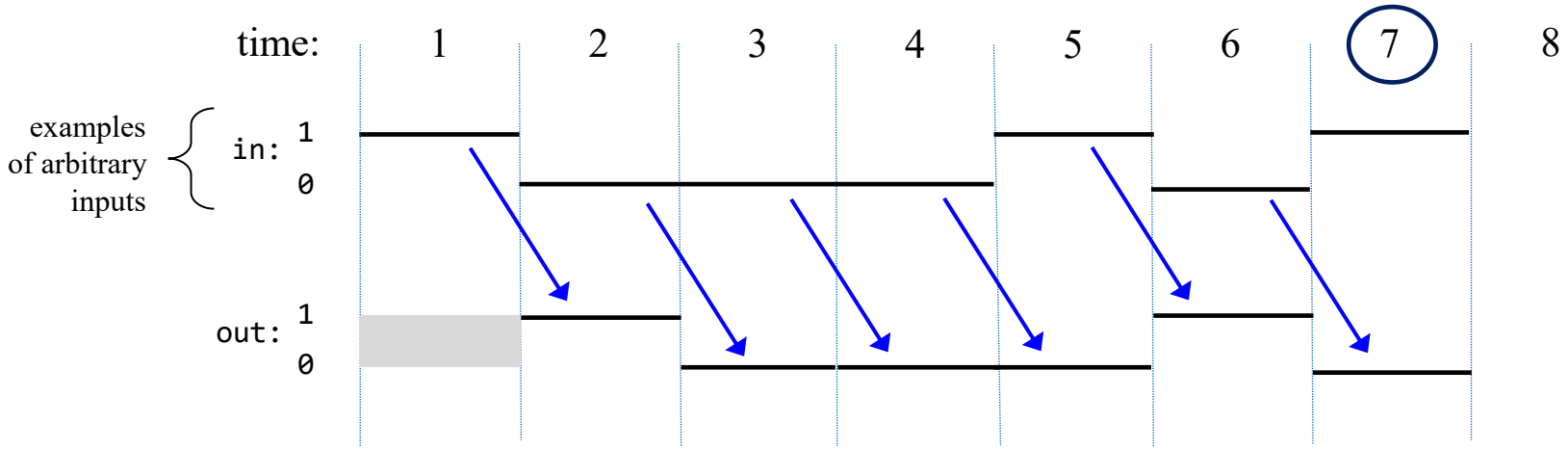
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



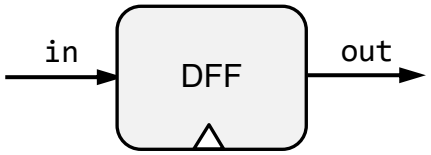
$\texttt{out}(t) = \texttt{in}(t-1)$



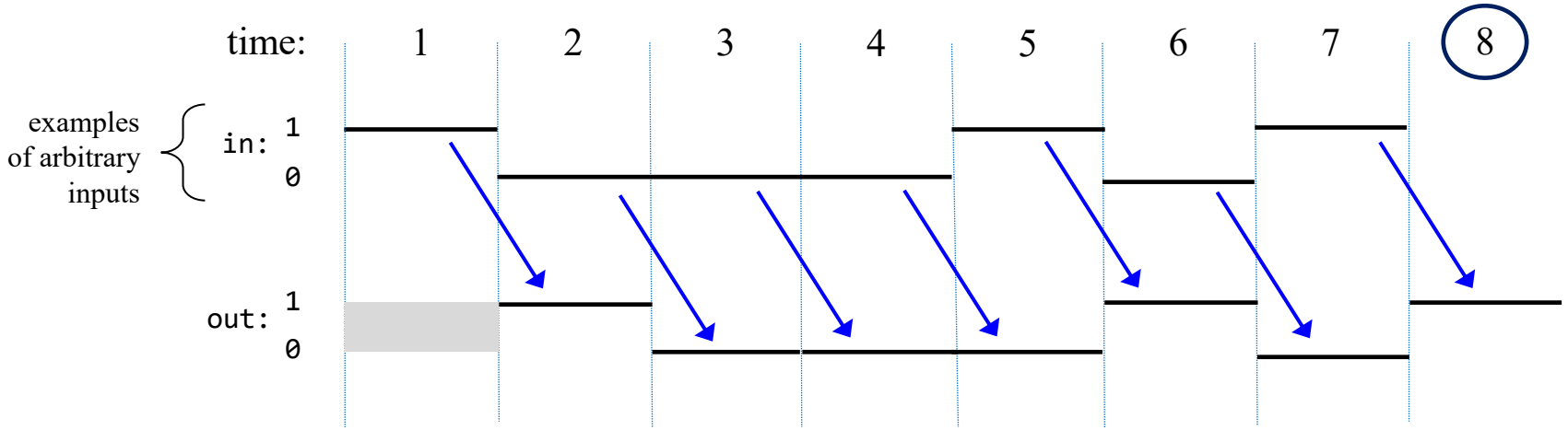
DFF (D Flip Flop)

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



$\texttt{out}(t) = \texttt{in}(t-1)$



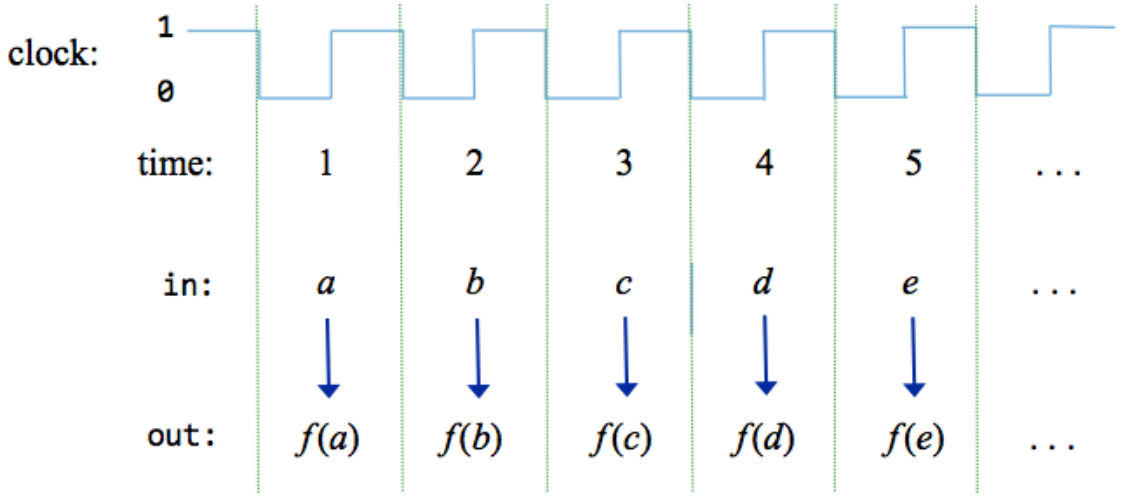
Combinational logic / Sequential logic

Combinational logic:

The output depends on the current inputs

The clock is used to stabilize outputs

时钟用于稳定输出，避免毛刺



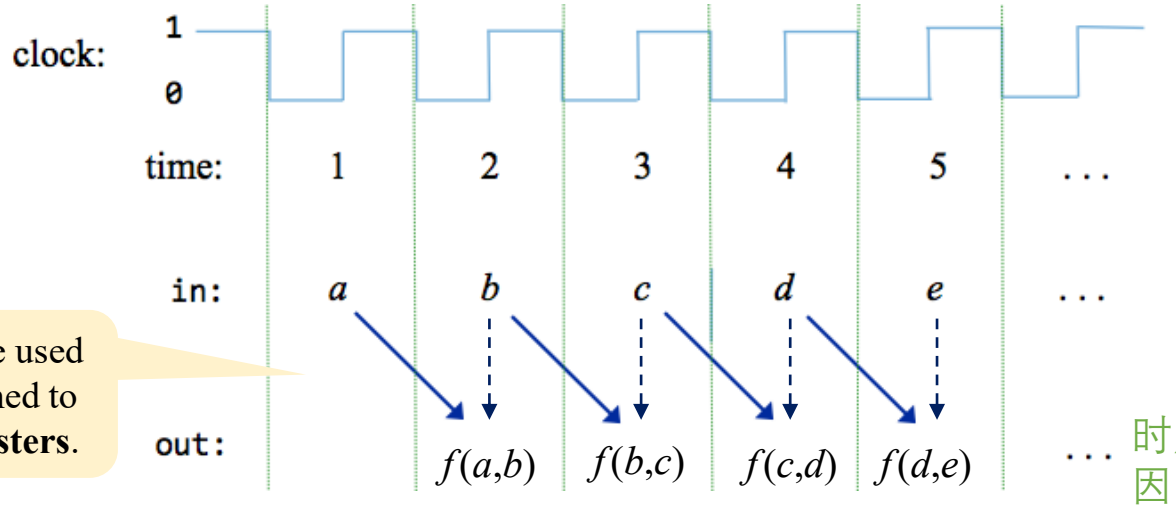
组合电路在同一个时钟周期内产生输出

Sequential logic:

The output depends on:

- Previous inputs
- Current inputs (optionally)

This behavior can be used to build chips designed to maintain state: **Registers**.



时序电路在下一个时钟周期内产生输出
因为要等待时钟上升/下降边缘触发

Combinational logic / Sequential logic

1. 组合逻辑（Combinational Logic）

核心特性：没有记忆，输出100%由当前时刻的输入决定，和电路之前的历史状态、过往输入完全无关。

> 类比：自动售货机
> 你当前投5元、按下「可乐」按钮，它就立刻出可乐，不管你之前投过多少钱、买过什么饮料，只要当前输入是「5元+选可乐」，输出永远是可乐，不会记得你上个月买过什么。

2. 时序逻辑（Sequential Logic）

核心特性：有记忆能力，输出不仅和当前输入有关，还和电路的历史状态、之前的输入序列有关。

> 类比：奶茶店的积分卡
> 你每买一杯奶茶积1分，满10杯送1杯。同样是「买一杯」这个输入，你之前累计3杯和累计9杯，输出（积分余额/是否触发赠送）是完全不一样的——因为它需要「记住」你之前的消费记录（历史状态），这就是时序逻辑的记忆能力。
> 而它的记忆核心，就是我们之前讲的D触发器：用D触发器把历史状态存下来，每次更新时结合当前输入计算新的状态。

组合逻辑的典型应用

所有「实时计算、不需要记历史」的场景都用组合逻辑：

- 基础单元：加法器、减法器、比较器、译码器（比如把4位二进制译码为16路输出，仅看当前4位输入）、多路选择器（MUX，根据当前选择信号切换输入输出）、编码器；
- 高阶模块：CPU里的ALU（算术逻辑单元，负责加减乘除、逻辑运算）、图像处理里的像素运算单元、通信模块的编解码电路。

时序逻辑的典型应用

所有「需要记忆、需要按步骤执行、需要同步」的场景都用时序逻辑：

- 基础单元：D触发器、寄存器、计数器、移位寄存器；
- 高阶模块：CPU里的寄存器堆、缓存（Cache）、有限状态机（FSM，比如交通灯控制器、通信协议的状态机、CPU的指令译码器）、跨时钟域同步电路、FIFO缓存、数字钟。

Combinational logic / Sequential logic

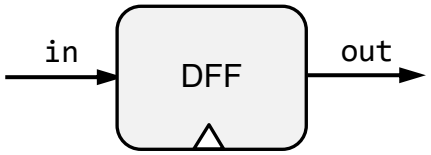
我们可以从6个关键维度明确两者的差异：

对比维度	组合逻辑	时序逻辑
记忆能力	无记忆，输出仅依赖当前输入	有记忆，输出=当前输入+历史状态
核心组成元件	仅用基本门电路（与/或/非/异或门等）	必须包含**存储单元**（D触发器、寄存器、锁存器）+门电路
时钟依赖	无需时钟，输入变化后输出立刻响应（忽略传输延迟）	通常需要全局时钟同步，仅在时钟边沿（上升/下降沿）更新状态
反馈回路合法性	禁止输出直接接回输入形成环路，会引发震荡/逻辑混乱	允许合法反馈（反馈信号通过存储单元锁存，比如计数器的输出接回输入实现累加）
抗干扰能力	易受输入毛刺影响，输出可能出现短暂错误脉冲（毛刺）	边沿采样抗干扰，只要满足建立/保持时间约束，就能稳定采样，不会出现毛刺
分析方法	真值表、布尔代数、卡诺图（穷举所有输入组合对应输出）	状态表、状态转换图、时序约束分析（需要同时考虑当前输入和历史状态）

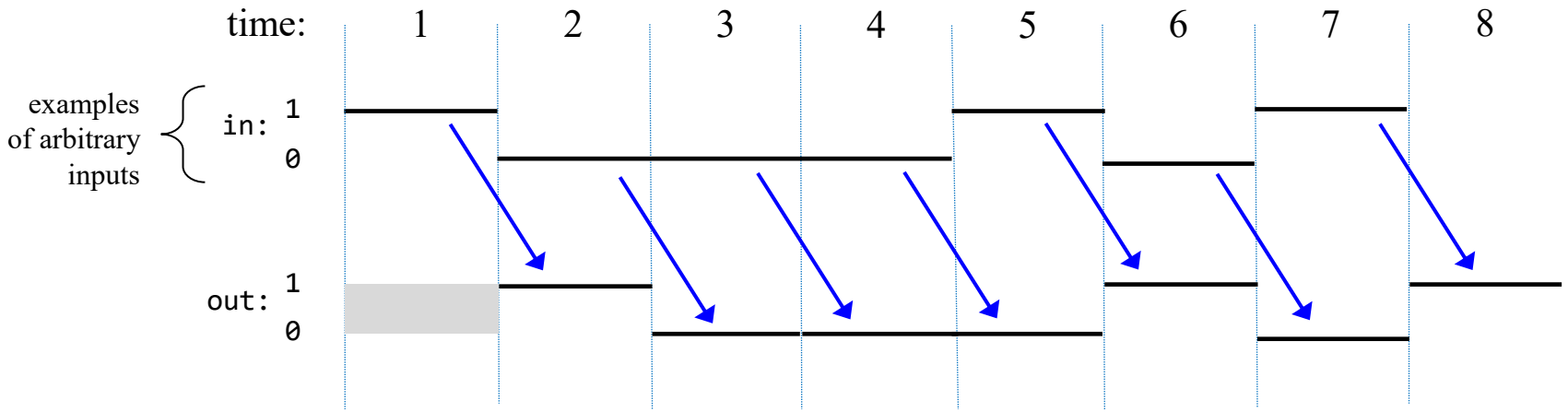
From DFF to a 1-bit register

Data Flip Flop (aka *latch*)

The most elementary sequential gate: Outputs the input in the previous time-step



$\texttt{out}(t) = \texttt{in}(t-1)$

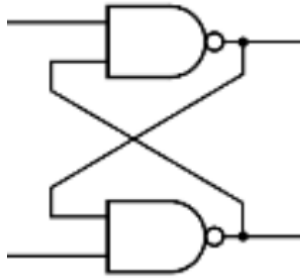


How can we "load" and then "maintain" a value (0 or 1) over time, without having to feed the value in every cycle?

我们如何随着时间的推移"加载"并"保持"一个值（0或1），而不必在每个周期中都输入该值?

DFF implementation

- 除其他事项外，DFF门逻辑实现还需要将Nand门与反馈回路连接起来
- 最终的实现是优雅的，但不可能在提供的硬件模拟器上实现
- 硬件模拟器不允许组合芯片（如Nand）的循环
- 因此，我们将使用内置的DFF实现：

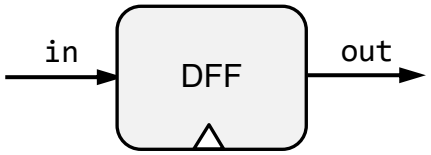


Part of a possible DFF implementation

```
/** Data Flip-flop: out(t) = in(t-1)
 *  where t is the current time unit. */
CHIP DFF {
    IN  in;
    OUT out;

    BUILTIN DFF;
    CLOCKED in;
}
```

From DFF to a 1-bit register

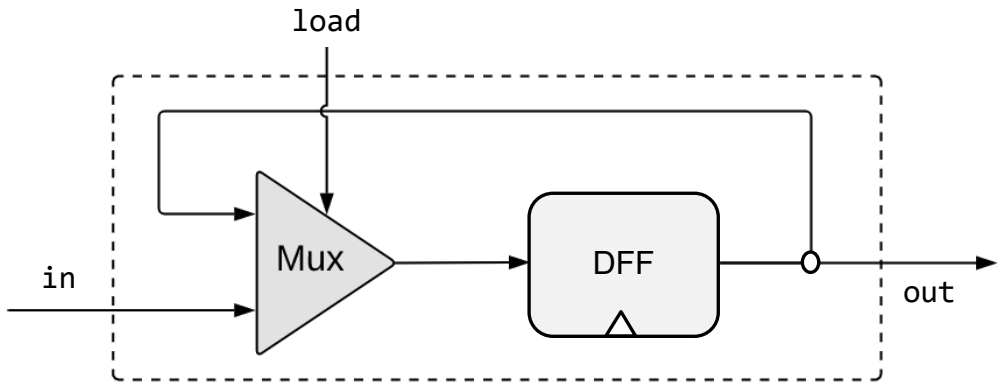


$\texttt{out}(t) = \texttt{in}(t-1)$

We have to realize a “loading” behavior and a ”storing” behavior,
and be able to select between these two states

How can we “load” and then “maintain” a value (0 or 1) over time,
without having to feed the value in every cycle?

From DFF to a 1-bit register



if load$(t-1)$ then
  out(t) = in$(t-1)$
else
  out(t) = out$(t-1)$

1-bit Register

Stores one bit over time

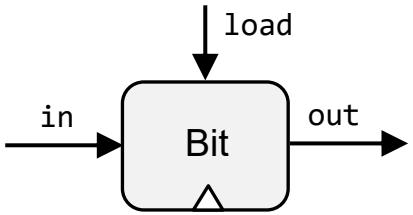
We have to realize a “loading” behavior and a ”storing” behavior, and be able to select between these two states

Behavior

if `load == 1` the register’s value becomes `in`

else the register maintains its current value

1-Bit Register



if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

1-Bit register CHIP的外部芯片视图

Usage:

To read:

probe out(检测out) (out always emits（发射） the register's state)

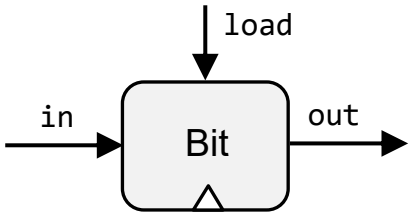
如果要读1-Bit register的value，直接检测Bit的out引脚即可

To write:

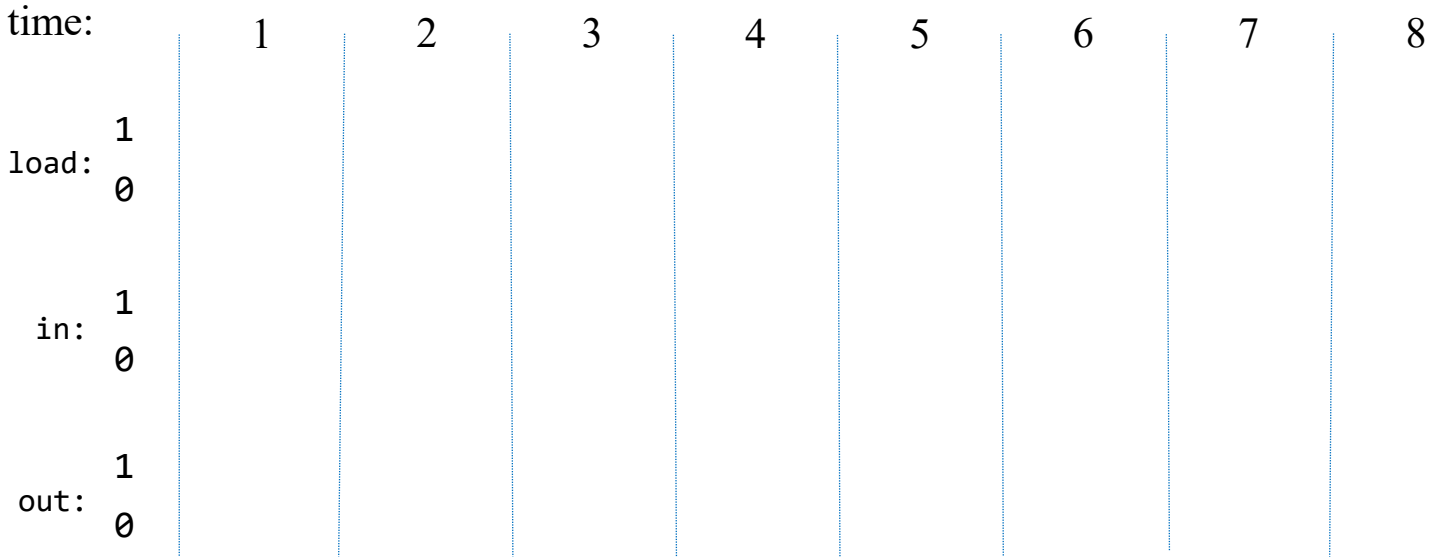
set in = v Result: The register's state becomes v;
set load = 1 From the next time-step onward, out will emit v

如果要向1-Bit register写入value(0/1)，需将load=1，in=v，
这样下个时钟周期Bit就会把v记录下来： out(t) = in(t – 1)

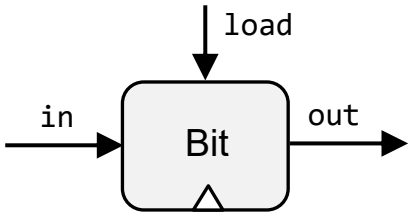
1-Bit Register



if load$(t-1)$ then out(t) = in$(t-1)$
else out(t) = out$(t-1)$

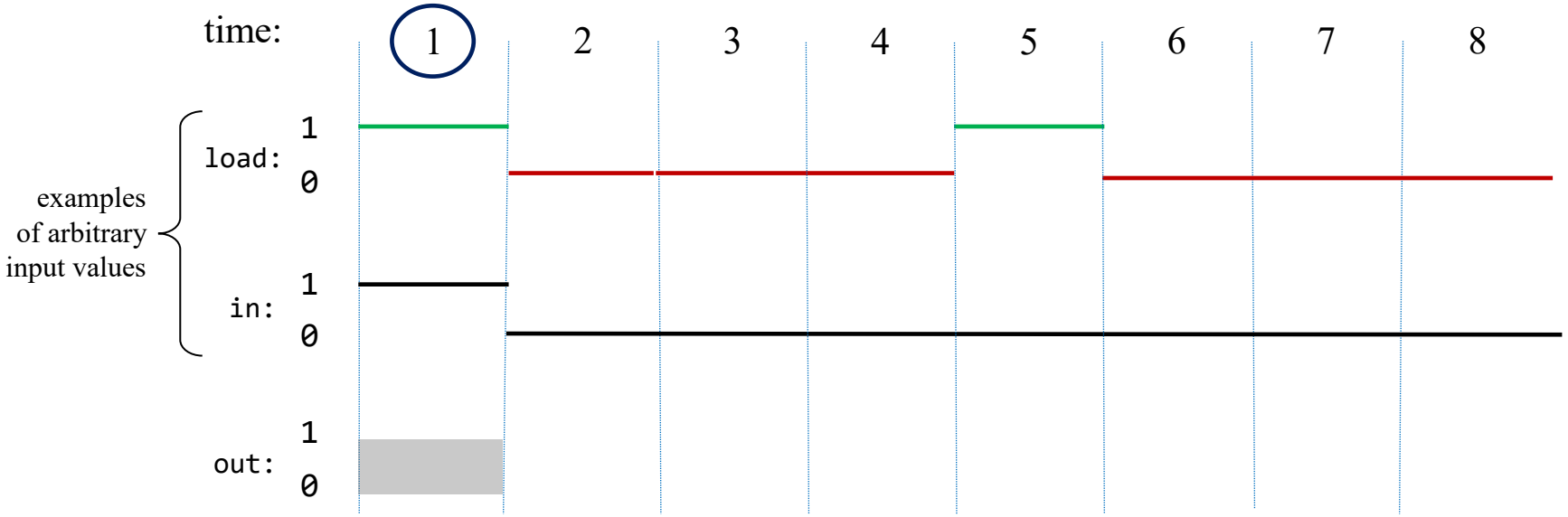


1-Bit Register

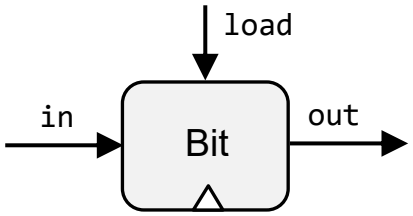


if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

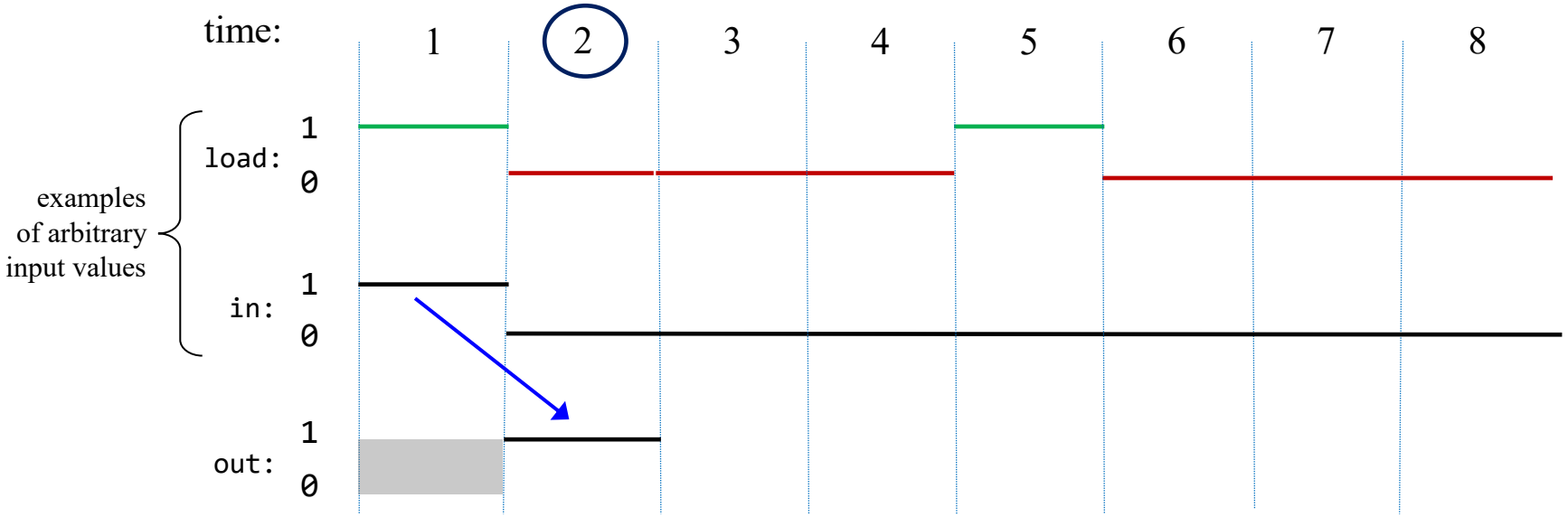
注意：虽然当前时间周期load=1，但是时序电路是在下个周期处理上个周期的值的



1-Bit Register

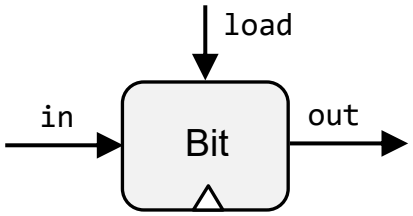


if load$(t-1)$ then out(t) = in$(t-1)$
else out(t) = out$(t-1)$

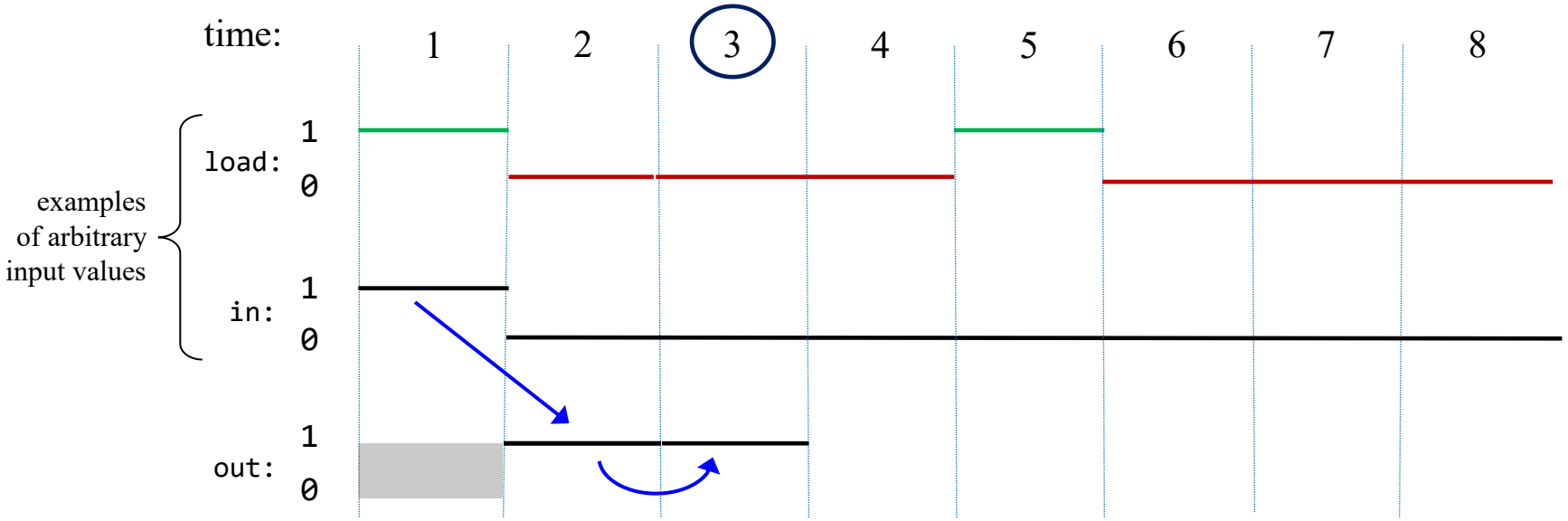


load = 1, in = 1 : out(t) = in$(t-1)$ =1

1-Bit Register

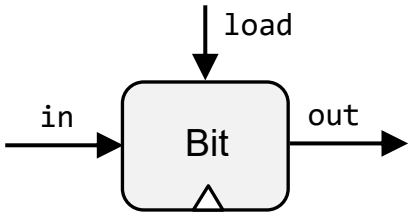


if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

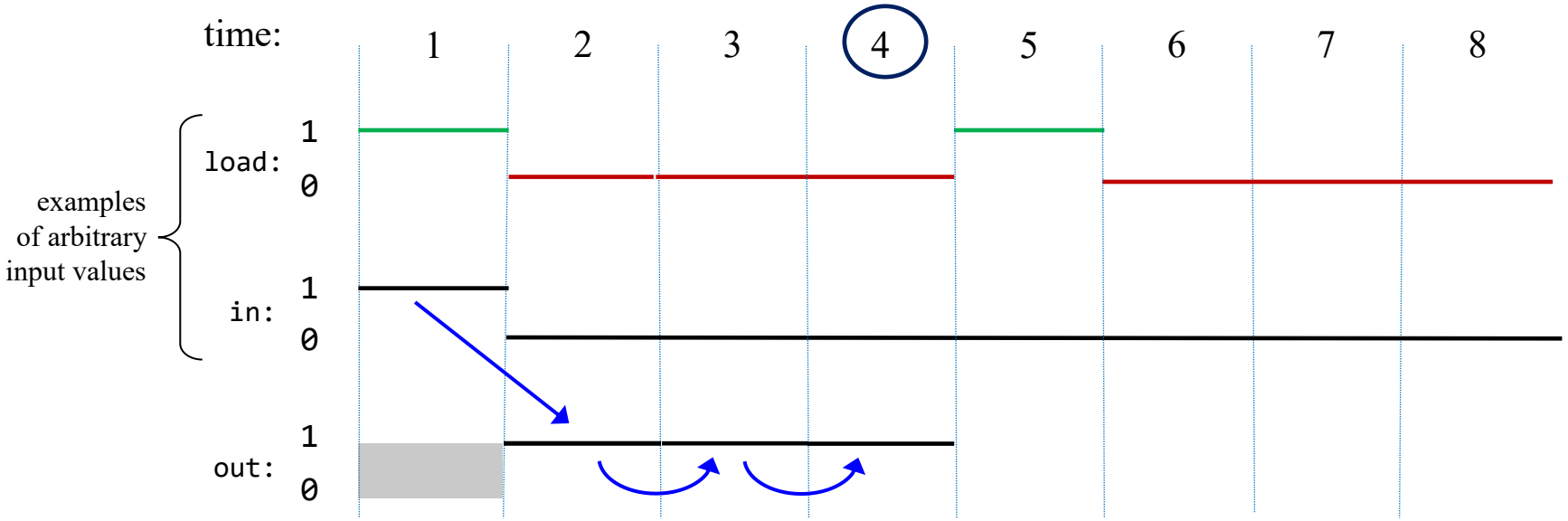


load = 0: out(t) = out($t-1$) =1

1-Bit Register

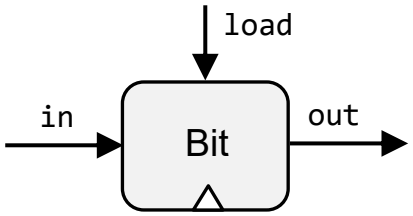


if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)



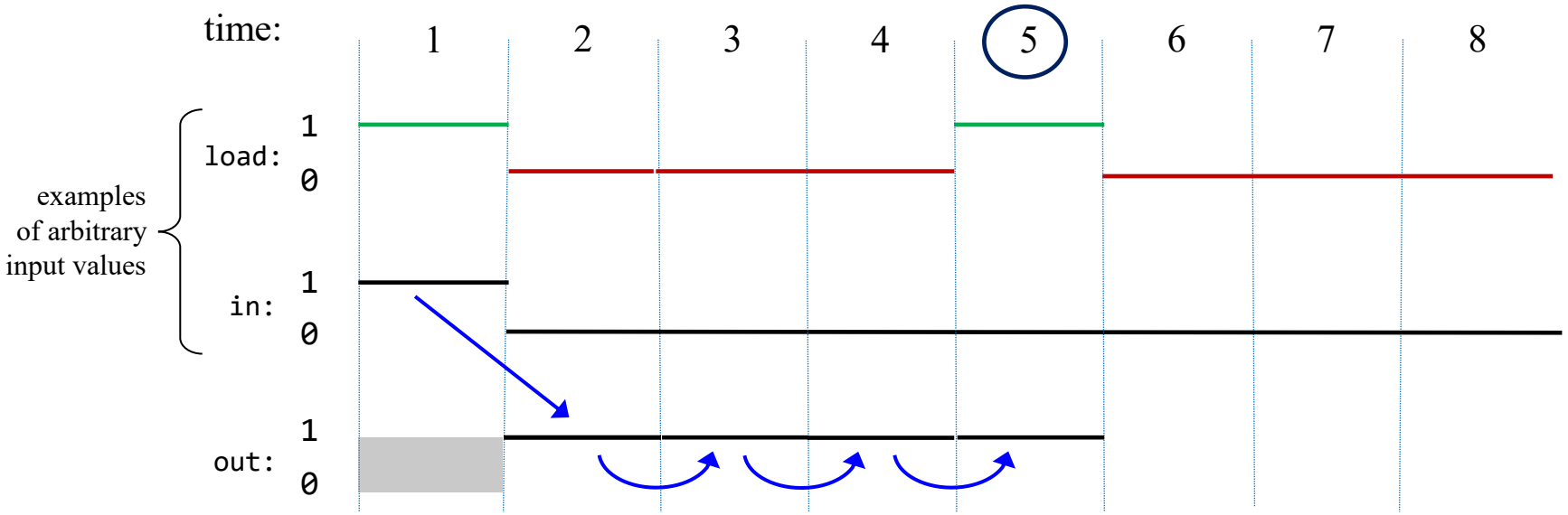
load = 0: out(t) = out($t-1$) =1

1-Bit Register



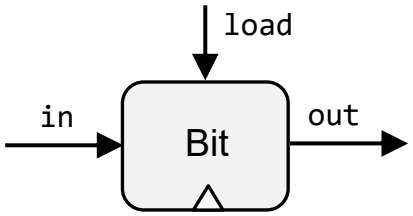
if load$(t-1)$ then out(t) = in$(t-1)$
else out(t) = out$(t-1)$

注意：虽然当前时间周期load=1，但是时序电路是在下个周期处理上个周期的值的

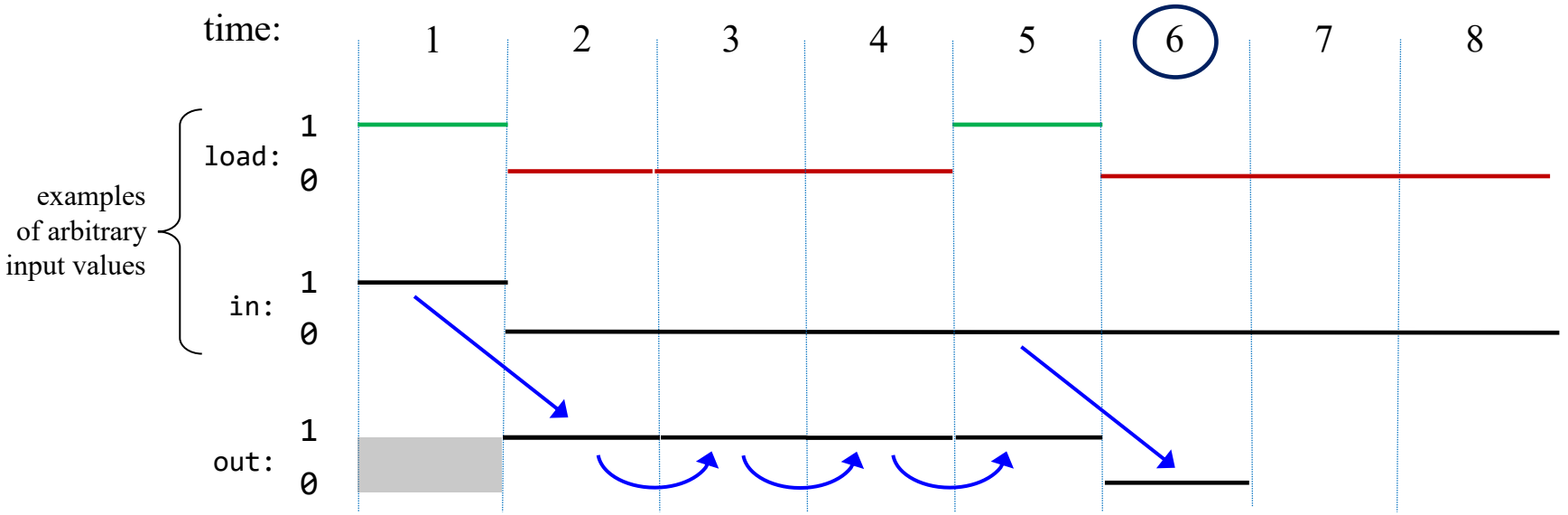


load = 0: out(t) = out$(t-1)$ =1

1-Bit Register

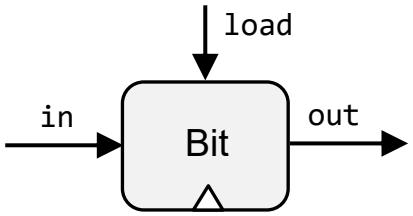


if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

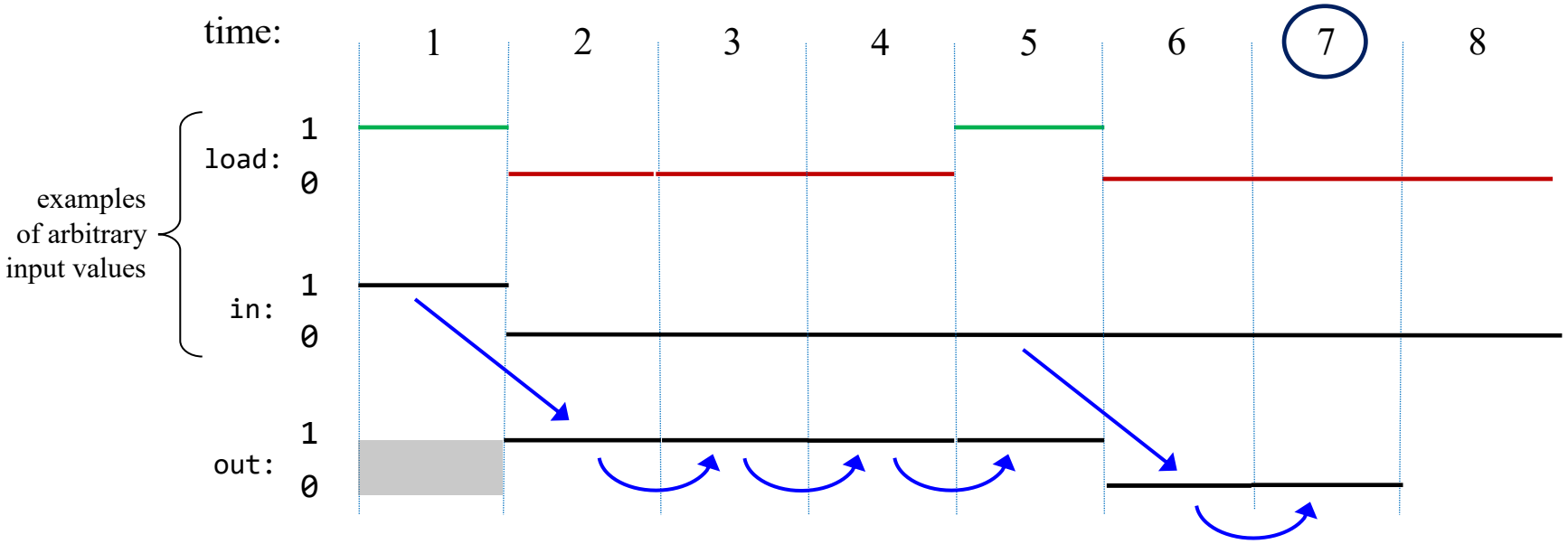


load = 1, in = 0 : out(t) = in($t-1$) = 0

1-Bit Register

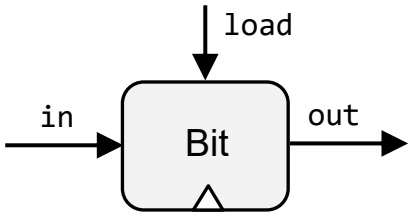


if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

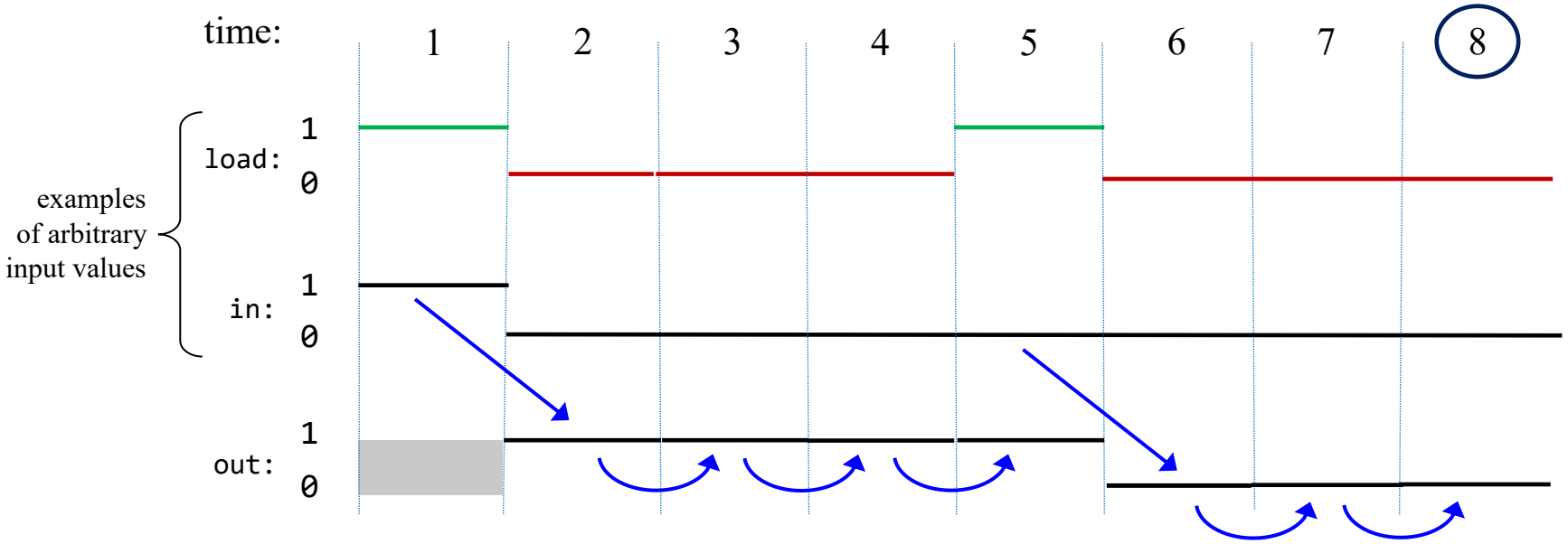


load = 0: out(t) = out($t-1$) =1

1-Bit Register

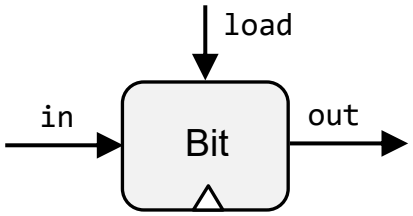


if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

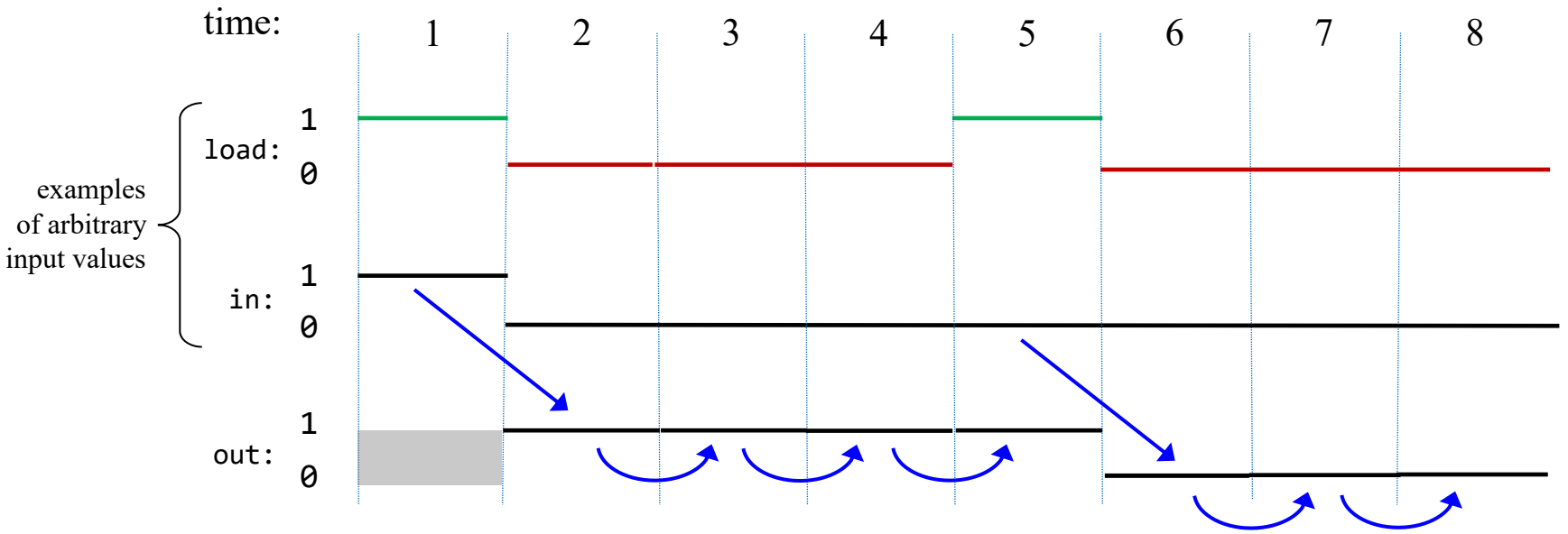


load = 0: out(t) = out($t-1$) =1

1-Bit Register

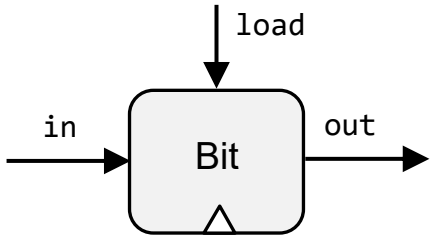


if load$(t-1)$ then out(t) = in$(t-1)$
else out(t) = out$(t-1)$



Resulting behavior: "loading" "storing" "loading" "storing"

1-bit Register

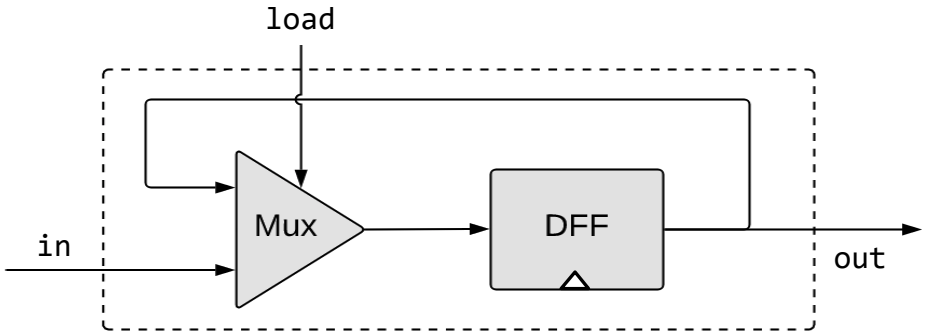


Bit.hdl

```
/** 1-bit register:
    if  load(t–1) then out(t) = in(t–1)
    else               out(t) = out(t–1))  */

CHIP Bit {
    IN in, load;
    OUT out;

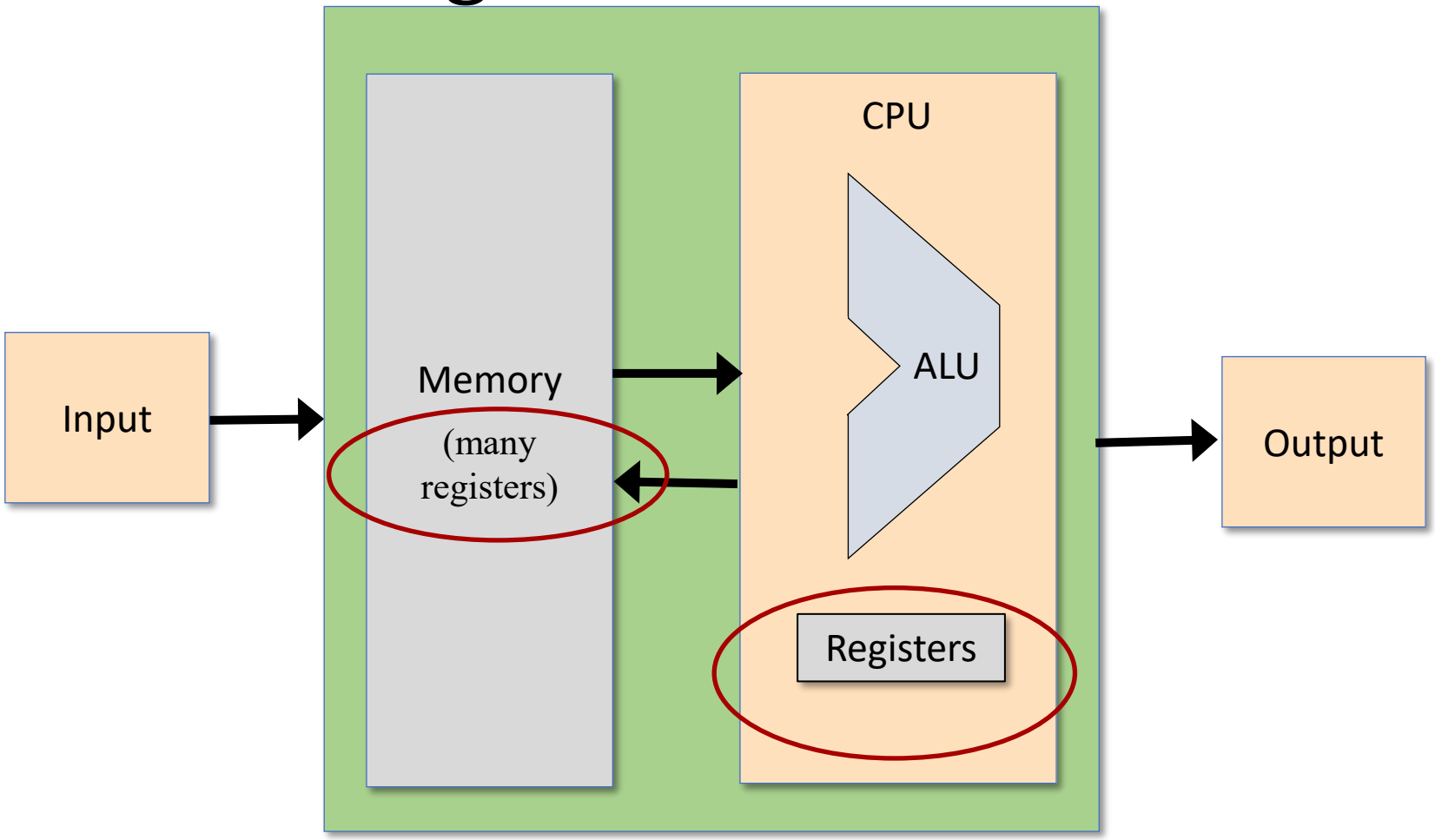
    PARTS:
     Mux(a=dffout, b=in, sel=load, out=muxout);
     DFF(in=muxout, out=dffout,out = out);
}
```



Implementation tip:

Follow the chip diagram

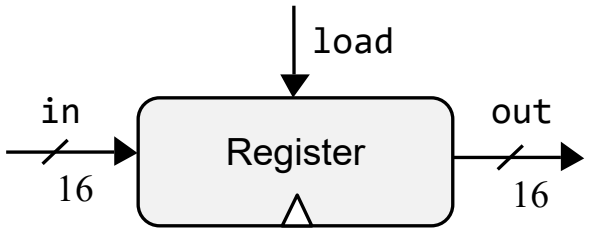
Multi-bit Registers



Multi- bit Registers

- A register is a collection of bits
- The width of a register is dependent on the architecture
  - Our architecture is 16-bit
- Share a common **load** control bit

16-bit Register



Register.tst

set in -321,
set load 1,
tick, // tick是低电平，Register不做处理
output; // output表格中，Register值不变

// tock是高电平，执行tock的瞬间就是时钟上升沿，会触发所有时序器件（DFF、Bit、寄存器等）采样输入、更新输出
tock,
output; // output表格中，Register值变成-321

Register.hdl

```
/** 1-bit register:
    if  load(t – 1) then out(t) = in(t – 1)
    else                 out(t) = out(t – 1))  */

CHIP Bit {
    IN in[16], load;
    OUT out[16];

    PARTS:
     Bit(in=in[0] , load= load, out=out[0] );
     Bit(in=in[1] , load= load, out=out[1] );
     Bit(in=in[2] , load= load, out=out[2] );
     Bit(in=in[3] , load= load, out=out[3] );


     Bit(in=in[4] , load= load, out=out[4] );
     Bit(in=in[5] , load= load, out=out[5] );
     Bit(in=in[6] , load= load, out=out[6] );
     Bit(in=in[7] , load= load, out=out[7] );


     Bit(in=in[8] , load= load, out=out[8] );
     Bit(in=in[9] , load= load, out=out[9] );
     Bit(in=in[10], load= load, out=out[10] );
     Bit(in=in[11], load= load, out=out[11] );


     Bit(in=in[12], load= load, out=out[12] );
     Bit(in=in[13], load= load, out=out[13] );
     Bit(in=in[14], load= load, out=out[14] );
     Bit(in=in[15], load= load, out=out[15] );


}
```

Counters

- We have discussed that the computer must keep track of which instruction should be fetched and executed next
- This task is regulated by a register typically called `Program Counter`
- We'll use the PC to store the address of the instruction that should be fetched and executed next
- The PC should support three abstractions:

Reset: fetch the first instruction	`PC = 0`
Next: fetch the next instruction	`PC++`
Goto: fetch instruction *n*	`PC =` *n*

Counters

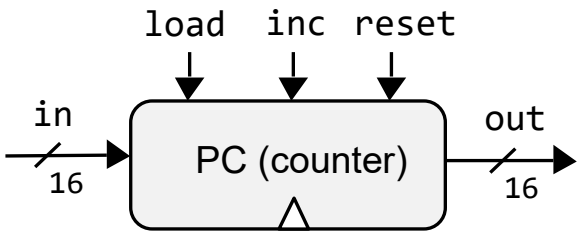
- Counters are a combination of combinatorial and sequential logic
- Counters implement
  - Reset
  - Increment by 1
  - Set to some specified value

```
PC = 0
```

```
PC++
```

```
PC = n
```

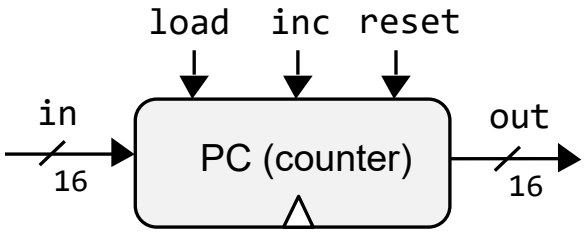
16-bit Counter(太复杂，不做要求)




```
/**
   A 16-bit counter with control bits.
   if       reset(t – 1)  out(t) = 0                 // resetting
   else if  load(t – 1)   out(t) = in(t – 1)         // setting
   else if  inc(t – 1)    out(t) = out(t – 1) + 1    // incrementing
   else                   out(t) = out(t – 1)        // maintaining
 */
CHIP PC {
   IN in[16], load, inc, reset;
   OUT out[16];
   PARTS:
    // Put your code here:
}
```

Implementation tip: Can be built from a Register, an Incrementer, and Mux's

Counter



if `reset`(t) `out`($t+1$) = `0`
else if `load`(t) `out`($t+1$) = `in`(t)
else if `inc`(t) `out`($t+1$) = `out`(t) + `1`
else `out`($t+1$) = `out`(t)

Usage:

To read:
probe `out`

To set:
set `in` to v,
assert `load`,
set the other control bits to `0`

To reset:
assert `reset`,
set the other control bits to `0`

To count:
assert `inc`,
set the other control bits to `0`

Summary

1. Introduced the concept of clocks & understand their use in computer systems
2. Discussed the difference between Sequential & Combinational logic
3. Looked at how Registers work and how they can be used to build memory and counters
4. Implement the circuits required to implement registers and Program Counter..